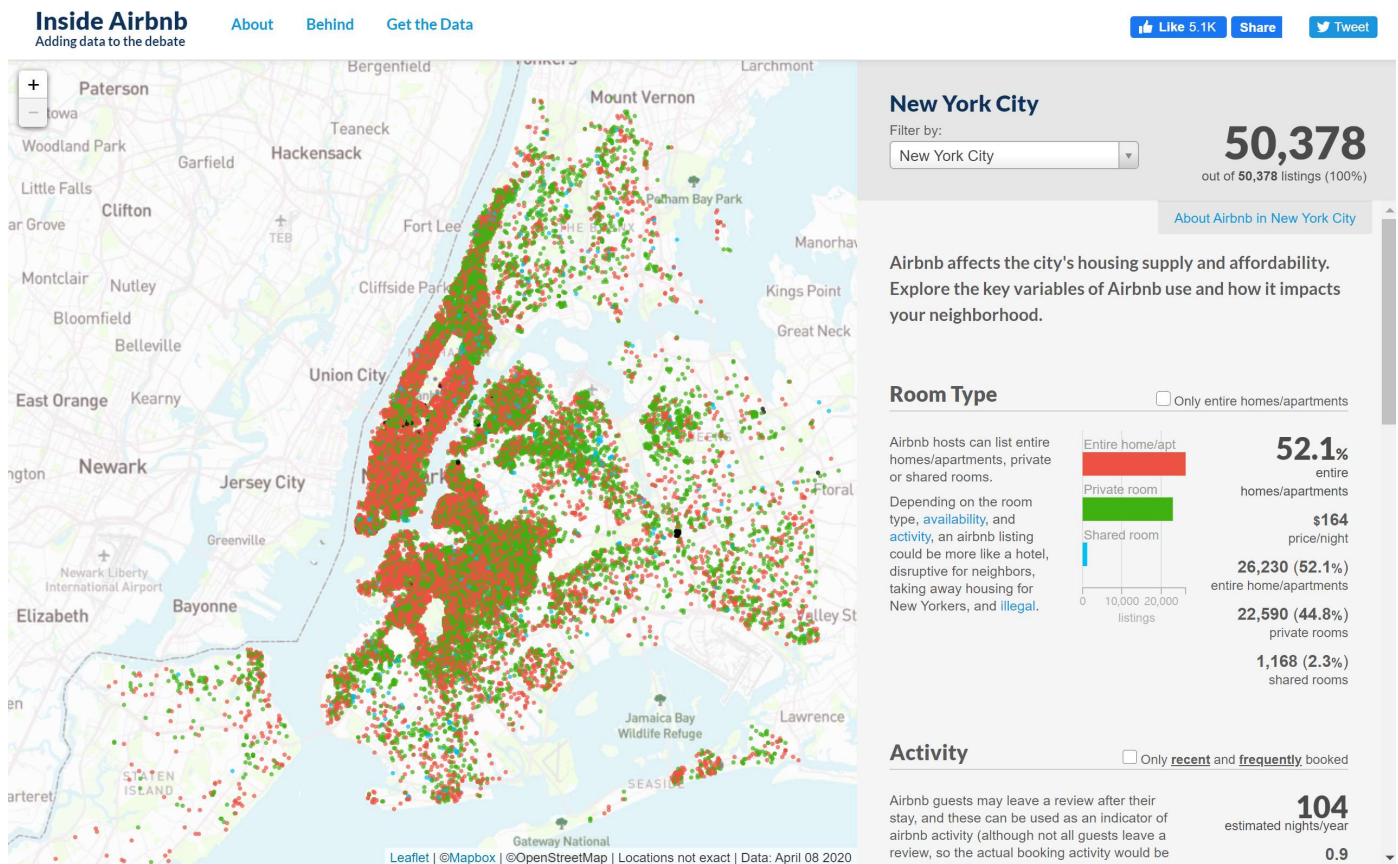


2019 年纽约市 Airbnb 租房价格预测

—— 可视计算基础大作业



数据来源: <http://insideairbnb.com/get-the-data.html>

1、摘要

对 Airbnb 提供的数百万套房源进行数据分析是公司的一个关键业务。Airbnb 数以百万计的清单生成了大量的数据——这些数据可以被分析和用于安全、业务决策、了解客户和供应商(房东)在平台上的行为和性能、指导营销计划、实施创新的附加服务等等。

这一份数据描述了 2019 年纽约市的 Airbnb 订单各类数据, 包括了 hosts, geographical availability, necessary metrics 等, 这里尝试用这些数据进行重要指标的预测(如未来的房租等)并给出相关的结论。

Airbnb 简介:

- Airbnb 成立于 2008 年 8 月, 总部设在美国加州旧金山市。
- Airbnb 是一个旅行房屋租赁社区, 用户可通过网络或手机应用程序发布、搜索度假房屋租赁信息并完成在线预定程序。据官网显示以及媒体报道, 其社区平台在 191 个国家、65,000 个城市为旅行者提供数以百万计的独特入住选择, 不管是公寓、别墅、城堡还是树屋。

- Airbnb 被时代周刊称为“住房中的 EBay”。

2、导入相关库

In [3]:

```
1 import numpy as np
2 import pandas as pd
3
4 import matplotlib
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 #import geopandas as gpd
8
9 import wordcloud
10 from wordcloud import WordCloud, STOPWORDS, ImageColorGenerator
11
12 import sklearn
13 from sklearn import preprocessing
14 from sklearn import metrics
15 from sklearn.metrics import r2_score, mean_absolute_error
16 from sklearn.preprocessing import OneHotEncoder, RobustScaler
17 from sklearn.ensemble import RandomForestRegressor
18 from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV, KFold, St
19 from sklearn.linear_model import Ridge, RidgeCV, Lasso, LassoCV, LinearRegression, HuberRegre
20
21 from xgboost import XGBRegressor
22 import xgboost as xgb
23 import graphviz
24
25 import scipy.stats as stats
26 from scipy.stats import norm
27 import statsmodels
28 import statsmodels.api as sm
29
30 import warnings
31
32 plt.style.use('fivethirtyeight')
33 matplotlib.rcParams['font.family'] = "Arial"
34 %matplotlib inline
35
36 warnings.filterwarnings("ignore")
37 colors=['#66CCCC', '#CCFF66', '#FF99CC', '#FF6666', '#FFFF66', '#0099CC']
```

3、数据准备

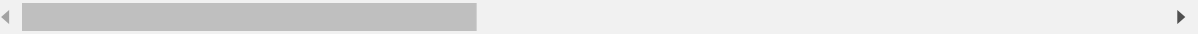
3.1 读取 csv 数据文件

In [4]:

```
1 data=pd.read_csv('AB_NYC_2019.csv')
2 data_origin=data
3 data.head()
```

Out [4]:

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	40.64749
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.75362
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	40.80902
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.68514
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.79851



3.2 初步统计

In [5]:

```
1 print('列数:', data.shape[0])
2 print('行数:', data.shape[1])
3 print('属性:\n', data.columns.tolist())
4 print('缺失值数目:', data.isnull().values.sum())
5 print('唯一值统计:')
6 print(data.nunique())
```

列数: 48895

行数: 16

属性:

['id', 'name', 'host_id', 'host_name', 'neighbourhood_group', 'neighbourhood', 'latitude', 'longitude', 'room_type', 'price', 'minimum_nights', 'number_of_reviews', 'last_review', 'reviews_per_month', 'calculated_host_listings_count', 'availability_365']

缺失值数目: 20141

唯一值统计:

id	48895
name	47905
host_id	37457
host_name	11452
neighbourhood_group	5
neighbourhood	221
latitude	19048
longitude	14718
room_type	3
price	674
minimum_nights	109
number_of_reviews	394
last_review	1764
reviews_per_month	937
calculated_host_listings_count	47
availability_365	366

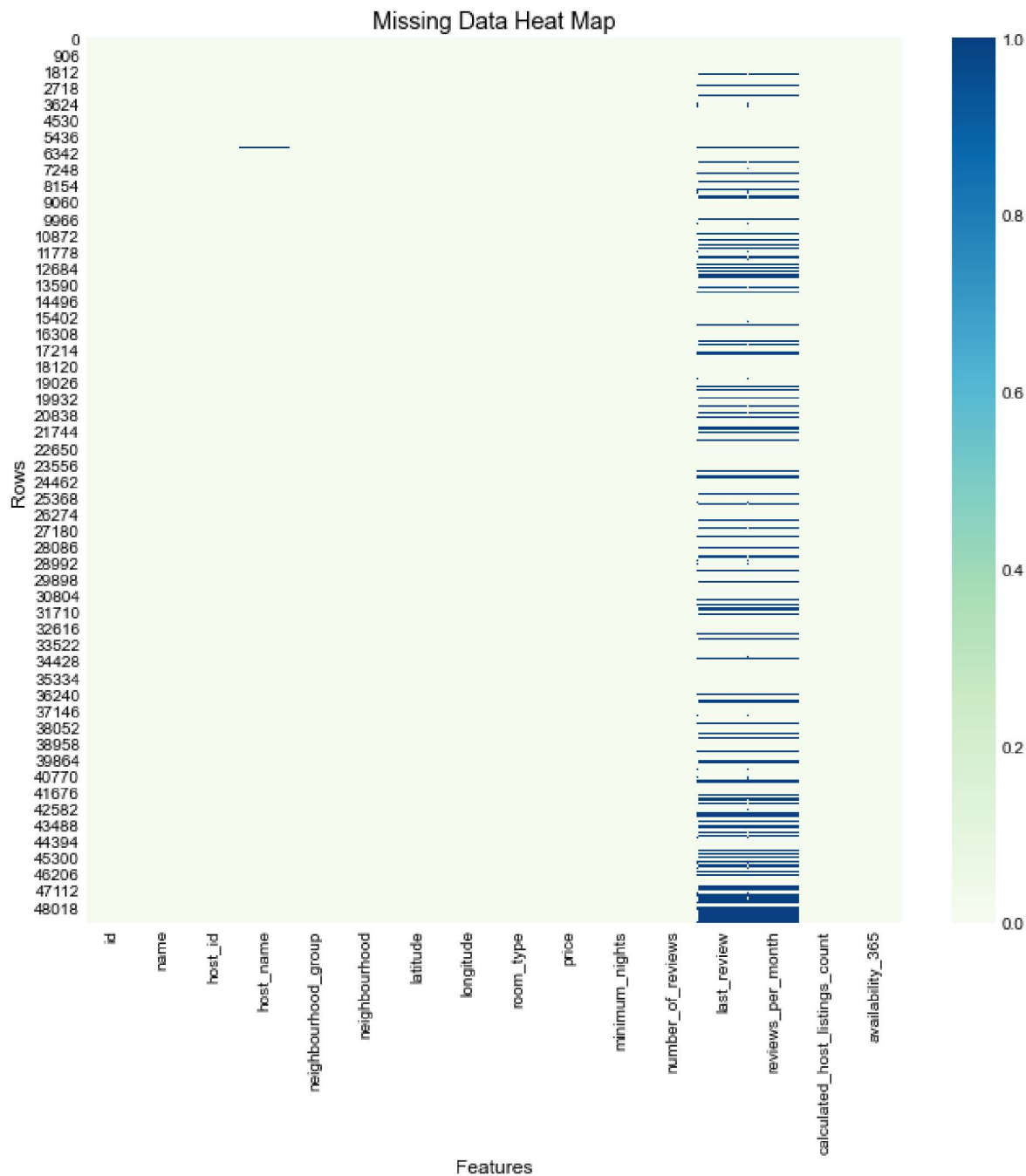
dtype: int64

3.3 缺失值分析

观察缺失值热力图

In [6]:

```
1 plt.figure(figsize=(10,10))
2 sns.heatmap(data.isnull(), cbar=True, cmap='GnBu')
3 plt.xlabel("Features")
4 plt.ylabel("Rows")
5 plt.title('Missing Data Heat Map')
6 plt.show()
```



说明：从缺失值的热力图可以看到，last_review 和 reviews_per_month 这两列数据的缺失情况较为严重，其中 last_review 为最新评论的日期，reviews_per_month 为每月的评论数，在构建预测模型时需考虑对其进行 drop 或者补全。

计算缺失值所占百分比

In [7]:

```
1 miss = data.isnull().sum().sort_values(ascending=True)
2 #print(total)
3 percent = miss/data.shape[0]*100
4 missing_data = pd.concat([miss, percent], axis=1, keys=['Miss', 'Percent'])
5 missing_data.head(16)
```

Out[7]:

	Miss	Percent
id	0	0.000000
host_id	0	0.000000
neighbourhood_group	0	0.000000
neighbourhood	0	0.000000
latitude	0	0.000000
longitude	0	0.000000
room_type	0	0.000000
price	0	0.000000
minimum_nights	0	0.000000
number_of_reviews	0	0.000000
calculated_host_listings_count	0	0.000000
availability_365	0	0.000000
name	16	0.032723
host_name	21	0.042949
last_review	10052	20.558339
reviews_per_month	10052	20.558339

说明：从这里我们可以看到 last_review 与 reviews_per_month 缺失的数目相同，约占了对应列所有数据的 20%，而 name 和 host_name 的数据略有缺失。

3.4 连续型数据的初步统计

In [8]:

```
1 data.describe().T
```

Out[8]:

	count	mean	std	min	25%
id	48895.0	1.901714e+07	1.098311e+07	2539.00000	9.471945e+06
host_id	48895.0	6.762001e+07	7.861097e+07	2438.00000	7.822033e+06
latitude	48895.0	4.072895e+01	5.453008e-02	40.49979	4.069010e+01
longitude	48895.0	-7.395217e+01	4.615674e-02	-74.24442	-7.398307e+01
price	48895.0	1.527207e+02	2.401542e+02	0.00000	6.900000e+01
minimum_nights	48895.0	7.029962e+00	2.051055e+01	1.00000	1.000000e+00
number_of_reviews	48895.0	2.327447e+01	4.455058e+01	0.00000	1.000000e+00
reviews_per_month	38843.0	1.373221e+00	1.680442e+00	0.01000	1.900000e-01
calculated_host_listings_count	48895.0	7.143982e+00	3.295252e+01	1.00000	1.000000e+00
availability_365	48895.0	1.127813e+02	1.316223e+02	0.00000	0.000000e+00

说明：从对连续数据的初步统计我们可以看到房租的平均价格在 152\$ 左右，最贵的房租达到了 10000。从 `minimum_nights` 这一行的统计数据可以发现大多数租客选择了一个星期的租期，而存在极端个例选择了 1250 天的租期，约为三年半的时间。

4、建立预测价格的简单模型

4.1 模型数据预处理

drop 无关属性

建立模型前需先对贡献不大的属性进行 drop，从预处理的分析中可以看到 `id`、`name`、`host_id`、`host_name` 属性的缺失值较少，但涵义为租客和房东的名称信息，为唯一值较多的离散值，可能对建模有一定的负面影响，这里简单建模考虑直接 drop 这四个属性。

In [9]:

```
1 data1=data.drop(columns=['host_id', 'id', 'name', 'host_name'], axis=1)
```

缺失值补全

`last_review`、`reviews_per_month` 属性的缺失值相对较多，如果直接 drop 会损失大量数据，应当考虑对其进行补全。同时，猜测 `last_review` 为 NaN 的原因是由于 `reviews_per_month` 也为 0 造成的，对这一猜测进行验证：

In [10]:

```
1 Nandata = data1[data1['reviews_per_month'].isnull()][['reviews_per_month', 'last_review']]
2 print(Nandata.shape)
3 print(Nandata.isnull().sum())
```

```
(10052, 2)
reviews_per_month    10052
last_review          10052
dtype: int64
```

可以看到 reviews_per_month 和 last_review 确实是对应成为缺失值，因此考虑用 0 补充 reviews_per_month 的缺失值，last_review 转换为时间戳后，缺失值补充为数据集内最早的评论时间。

In [11]:

```
1 data1['reviews_per_month'] = data1['reviews_per_month'].fillna(0)
2 data1['last_review'] = pd.to_datetime(data1['last_review'], infer_datetime_format=True)
3 earliest = min(data1['last_review'])
4 data1['last_review'] = data1['last_review'].fillna(earliest)
5 data1.head(5)
```

Out[11]:

	neighbourhood_group	neighbourhood	latitude	longitude	room_type	price	minimum_nigh
0	Brooklyn	Kensington	40.64749	-73.97237	Private room	149	
1	Manhattan	Midtown	40.75362	-73.98377	Entire home/apt	225	
2	Manhattan	Harlem	40.80902	-73.94190	Private room	150	
3	Brooklyn	Clinton Hill	40.68514	-73.95976	Entire home/apt	89	
4	Manhattan	East Harlem	40.79851	-73.94399	Entire home/apt	80	

确认数据集是否补全缺失:

In [12]:

```
1 miss = data1.isnull().sum().sort_values(ascending=True)
2 #print(total)
3 percent = miss/data1.shape[0]*100
4 missing_data = pd.concat([miss, percent], axis=1, keys=['Miss', 'Percent'])
5 missing_data.head(16)
```

Out[12]:

	Miss	Percent
neighbourhood_group	0	0.0
neighbourhood	0	0.0
latitude	0	0.0
longitude	0	0.0
room_type	0	0.0
price	0	0.0
minimum_nights	0	0.0
number_of_reviews	0	0.0
last_review	0	0.0
reviews_per_month	0	0.0
calculated_host_listings_count	0	0.0
availability_365	0	0.0

标称数据数值化

现在数据内仍然存有非数值性属性，需要对其进行数值化，有 `neighbourhood_group`、`neighbourhood`、`room_type` 和 `last_review` 四种非数值型属性，前三者可考虑用 `OneHotEncoder` 进行独热编码，后一种用减去数据集内最早的时间作为数值。

In [13]:

```
1 data1['last_review'] = data1['last_review'].apply(lambda x:x.toordinal() - earliest.toordinal())
2 data1_ob=data1.select_dtypes(include=['object'])
3 print(data1_ob.columns)
4 data1_ob_oh=pd.get_dummies(data1_ob)
5 data1_ob_oh.head(5)
```

Index(['neighbourhood_group', 'neighbourhood', 'room_type'], dtype='object')

Out[13]:

	neighbourhood_group_Bronx	neighbourhood_group_Brooklyn	neighbourhood_group_Manhattan
0	0	1	0
1	0	0	1
2	0	0	0
3	0	1	0
4	0	0	1

5 rows × 229 columns

构建预测房租价格的数据集

In [14]:

```
1 data1_num= data1.select_dtypes(exclude=['object'])
2 y = data1_num.loc[:, 'price']
3 data1_num.drop(columns=['price'], axis=1, inplace=True)
4 data1_pre = pd.concat([data1_num, data1_ob_oh, y], axis=1)
5 print(data1_pre.shape)
6 data1_pre.columns.tolist()
```

(48895, 238)

4.2 建立模型进行预测房租

利用五折验证的方法测试多个模型

In [15]:

```
1 X=data1_pre.iloc[:, :-1]
2 y = data1_pre.iloc[:, -1]
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
4 print(f'The shape of Xtrain: {X_train.shape}')
5 print(f'The shape of Xtest: {X_test.shape}')
6 print(f'The shape of ytrain: {y_train.shape}')
7 print(f'The shape of ytest: {y_test.shape}')
8 scaler = RobustScaler()
9 X_train = scaler.fit_transform(X_train)
10 X_test = scaler.fit_transform(X_test)
```

The shape of Xtrain: (39116, 237)

The shape of Xtest: (9779, 237)

The shape of ytrain: (39116,)

The shape of ytest: (9779,)

In [16]:

```
1 def r2_score(model):
2     return cross_val_score(model, X_train, y_train, cv=5)
3 def rmse_score(model):
4     return cross_val_score(model, X_train, y_train, scoring='neg_mean_squared_error', cv=5)
5
6 Model=[LinearRegression, Ridge, Lasso, HuberRegressor, RandomForestRegressor, XGBRegressor]
7 lname1=[]
8 lr21=[]
9 lrmse1=[]
10
11 for model in Model:
12     if model == XGBRegressor:
13         m=XGBRegressor(objective='reg:squarederror')
14     else:
15         m=model()
16     cv_r2=r2_score(m)
17     cv_rmse_score = rmse_score(m)
18     print('The mean R^2 of {} is: {:.3e}'.format(model.__name__, cv_r2.mean()))
19     print('The mean mse of {} is: {:.3e}'.format(model.__name__, -cv_rmse_score.mean()))
20     lname1.append(model.__name__)
21     lr21.append(cv_r2.mean())
22     lrmse1.append(-cv_rmse_score.mean())
```

The mean R² of LinearRegression is:-2.584e+13

The mean mse of LinearRegression is: 1.671e+18

The mean R² of Ridge is:1.213e-01

The mean mse of Ridge is: 5.431e+04

The mean R² of Lasso is:1.095e-01

The mean mse of Lasso is: 5.494e+04

The mean R² of HuberRegressor is:8.044e-02

The mean mse of HuberRegressor is: 5.660e+04

The mean R² of RandomForestRegressor is:7.672e-02

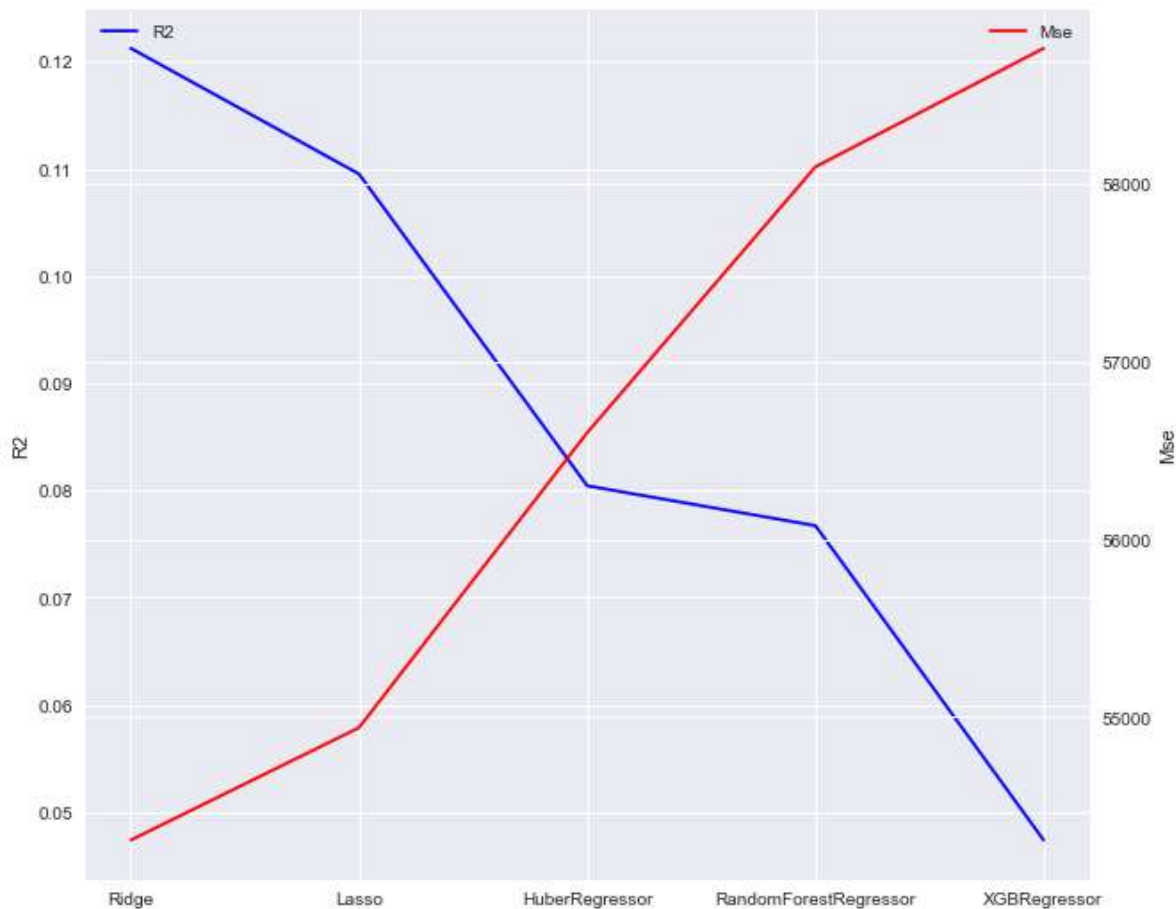
The mean mse of RandomForestRegressor is: 5.810e+04

The mean R² of XGBRegressor is:4.739e-02

The mean mse of XGBRegressor is: 5.876e+04

In [17]:

```
1 plt.style.use('seaborn')
2 fig = plt.figure(figsize=(9,9))
3 ax1 = fig.add_subplot(111)
4 ax1.plot(lname1[1:], lr21[1:], 'b', label="R2");
5 ax1.legend(loc=2)
6 ax1.set_ylabel('R2');
7 ax2 = ax1.twinx() # this is the important function
8 ax2.plot(lname1[1:], lrmsel[1:], 'r', label = "Mse")
9 ax2.legend(loc=1)
10 # ax2.set_xlim([0, np.e]);
11 ax2.set_ylabel('Mse');
12 plt.show()
```



说明：可以看到经过简单处理的数据在模型表现上并不好，因此接下来需要对各属性进行可视化分析，进行特征工程，筛选处理较优特征。

5、可视化与特征工程

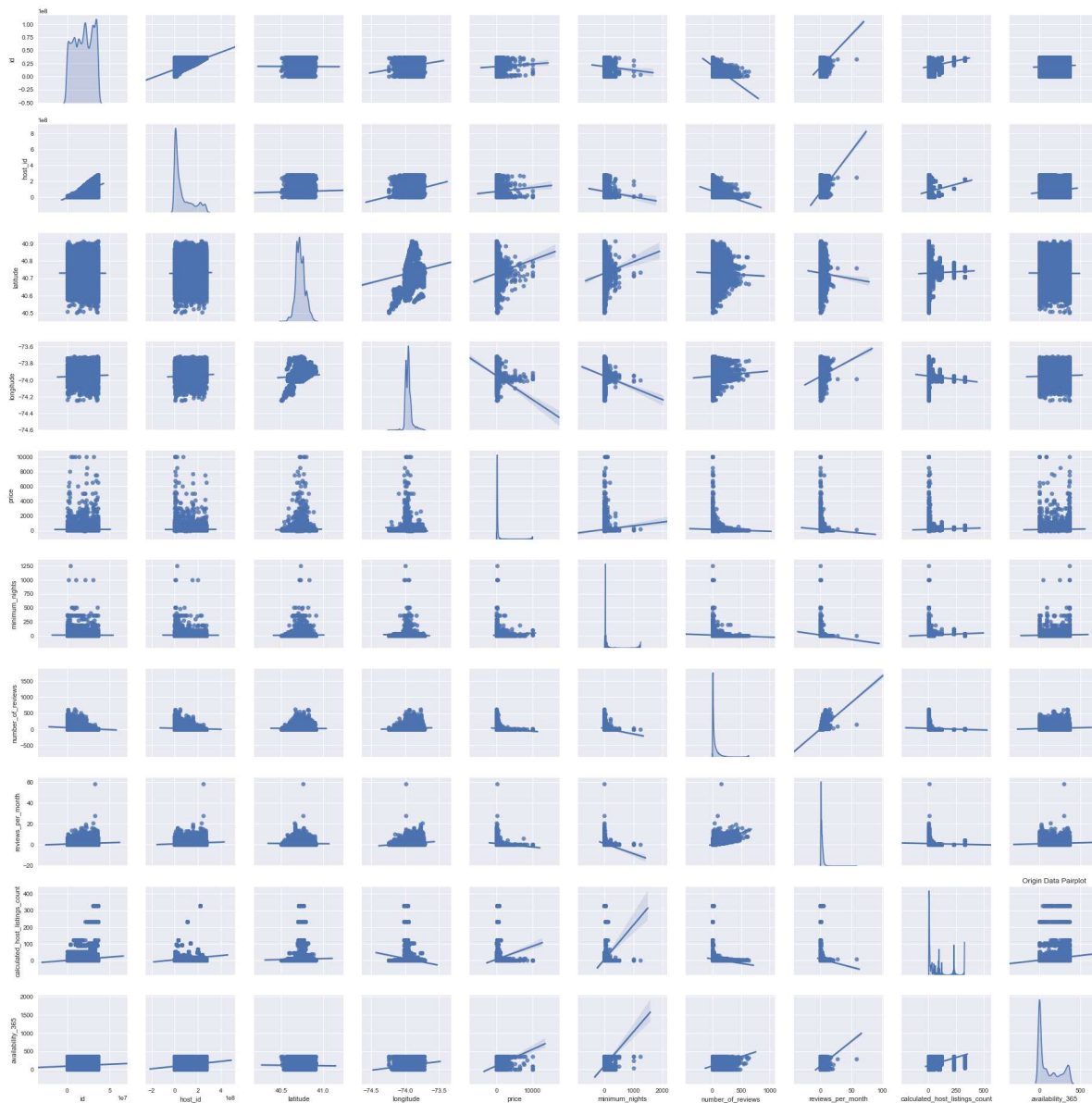
特征工程需要对每类属性进行分析，这里重新获取源数据对他们进行可视化分析

In [16]:

```
1 data_origin['reviews_per_month'] = data_origin['reviews_per_month'].fillna(0)
2 data_origin['last_review'] = pd.to_datetime(data_origin['last_review'], infer_datetime_format=True)
3 earliest = min(data_origin['last_review'])
4 data_origin['last_review'] = data_origin['last_review'].fillna(earliest)
5 data2=data_origin # data_origin为原始数据，data2为进行特征工程后的结果
```

In [17]:

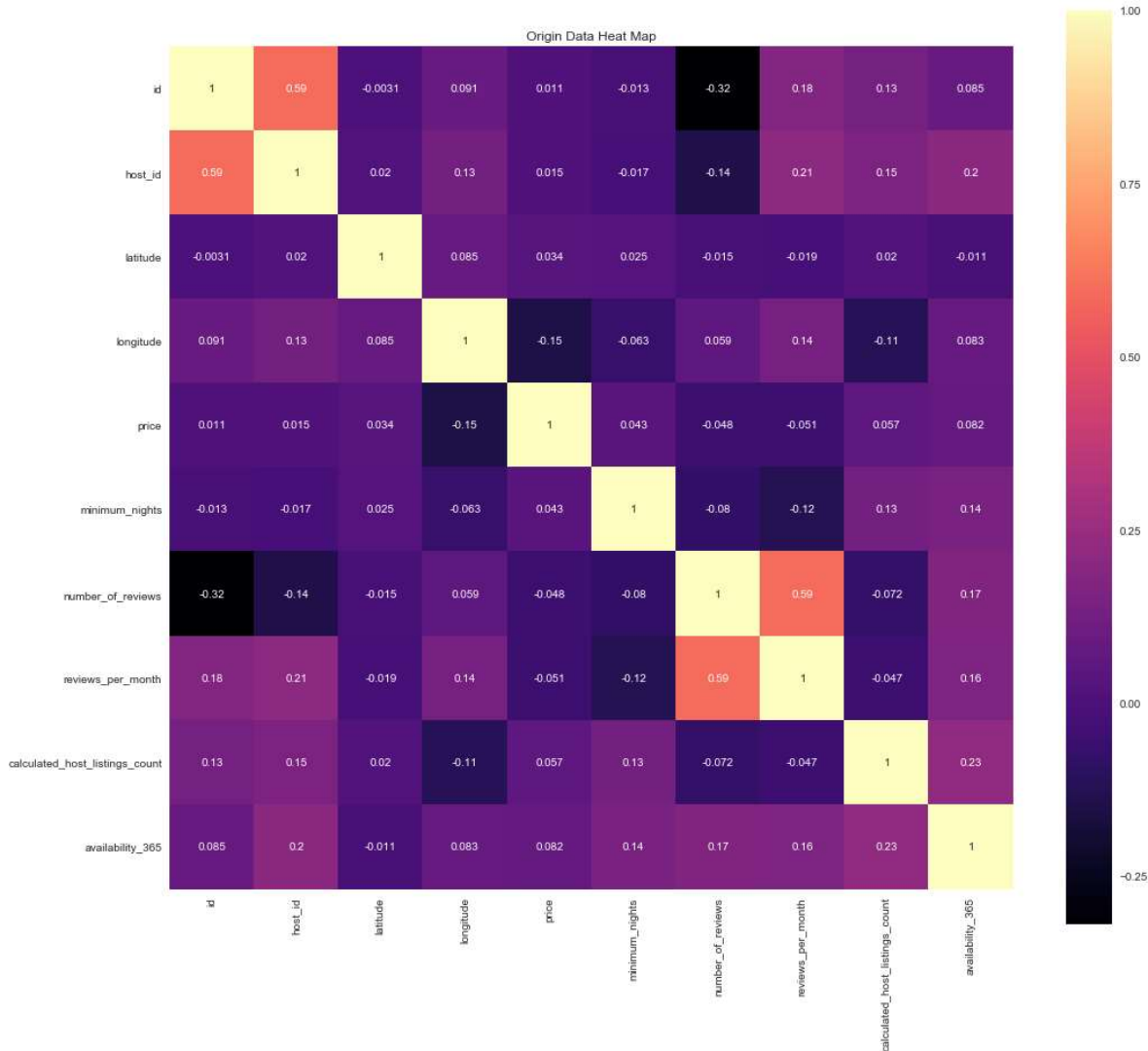
```
1 sns.pairplot(data_origin, kind="reg", diag_kind="kde", diag_kws=dict(shade=True))
2 plt.title('Origin Data Pairplot')
3 plt.show()
```



通过原始数据的 pairlot 可以发现，大部分数据的关联性较差，回归线难以经过大多数数据点，这可能会使得建模能力较差。

In [18]:

```
1 plt.style.use('seaborn')
2 corrmatrix = data_origin.corr()
3 f, ax = plt.subplots(figsize=(15,15))
4 sns.heatmap(corrmatrix, vmax=1, square=True, annot=True, cmap='magma')
5 plt.title('Origin Data Heat Map')
6 plt.show()
```



说明：各属性间的相关系数热力图也很好地印证了这一点，大部分的颜色较深，相关性较差。

5.1 Price

首先应当观察的需要进行预测的属性 `price`，这里绘制了各个 `neighbourhood_group` 的 `price` 分布。

In [19]:

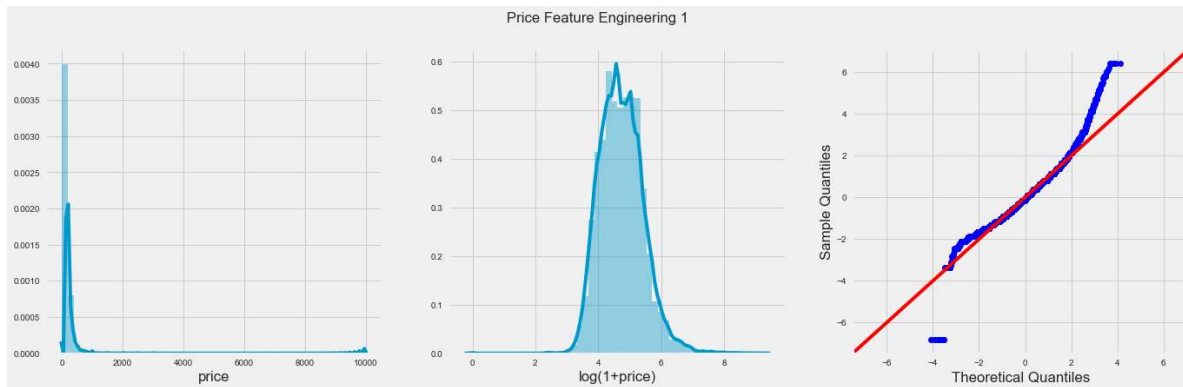
```
1 plt.style.use('fivethirtyeight')
2 fig, axes = plt.subplots(1,2, figsize=(20,7))
3 for i in [0,1]:
4     sns.distplot(data_origin[data_origin.neighbourhood_group=='Manhattan'].price, color=colors[0], hist=True)
5     sns.distplot(data_origin[data_origin.neighbourhood_group=='Brooklyn'].price, color=colors[1], hist=True)
6     sns.distplot(data_origin[data_origin.neighbourhood_group=='Queens'].price, color=colors[2], hist=True)
7     sns.distplot(data_origin[data_origin.neighbourhood_group=='Staten Island'].price, color=colors[3], hist=True)
8     sns.distplot(data_origin[data_origin.neighbourhood_group=='Bronx'].price, color=colors[4], hist=True)
9     if i == 1:
10         axes[1].set_title('Price Distribution for Price<2000')
11         plt.xlim(0,2000)
12     else:
13         axes[0].set_title('Price Distribution for All Price')
14 fig.suptitle('Price Distribution')
15 plt.show()
16
```



说明：从价格的分布图可以看出 Manhattan 这一区域的价位相对较高，各区域的价格分布有一定的集中倾向，这可能是大致模型预测效果不好的原因之一。这里尝试绘制原价格分布与对数变换后的结果进行比较（数据挖掘常用对数变换来改变原分布使其靠近正态分布），qqplot 显示了直观分布差异结果。

In [20]:

```
1 plt.style.use('fivethirtyeight')
2 fig, axes = plt.subplots(1,3, figsize=(21,6))
3 sns.distplot(data_origin['price'],color=colors[5], ax=axes[0])
4 sns.distplot(np.log1p(data_origin['price']),color=colors[5], ax=axes[1])
5 axes[1].set_xlabel('log(1+price)')
6 sm.qqplot(np.log1p(data_origin['price']), stats.norm, fit=True, line='45', ax=axes[2]);
7
8 fig.suptitle('Price Feature Engineering 1')
9 plt.show()
```



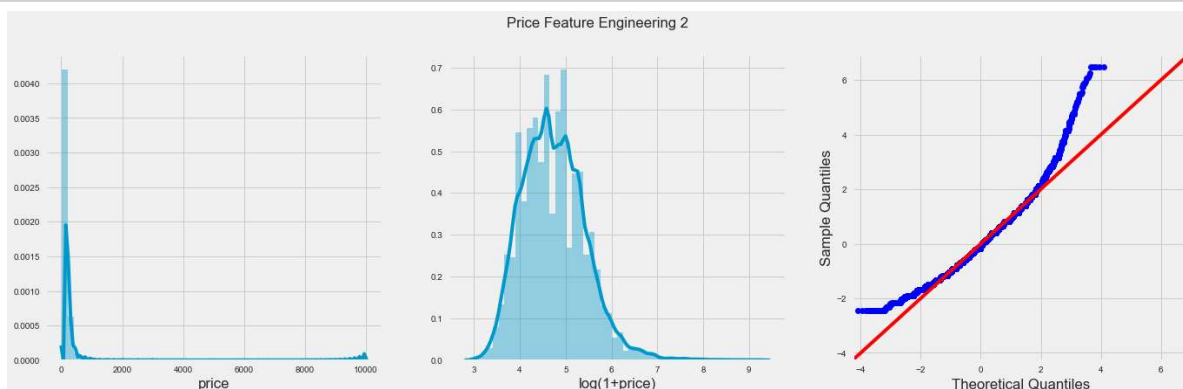
说明：可以看到对数处理后，数据的分布有一段接近正态分布了，但是仍有许多离群值导致分布比较倾斜，这里尝试先去除一些离群值（outlier）。

In [21]:

```
1 data2 = data_origin[np.log1p(data_origin['price']) < 8]
2 data2 = data_origin[np.log1p(data_origin['price']) > 3]
```

In [22]:

```
1 fig, axes = plt.subplots(1,3, figsize=(21,6))
2 sns.distplot(data2['price'],color=colors[5], ax=axes[0])
3 sns.distplot(np.log1p(data2['price']),color=colors[5], ax=axes[1])
4 axes[1].set_xlabel('log(1+price)')
5 sm.qqplot(np.log1p(data2['price']),stats.norm, fit=True, line='45', ax=axes[2]);
6 fig.suptitle('Price Feature Engineering 2')
7 plt.show()
```



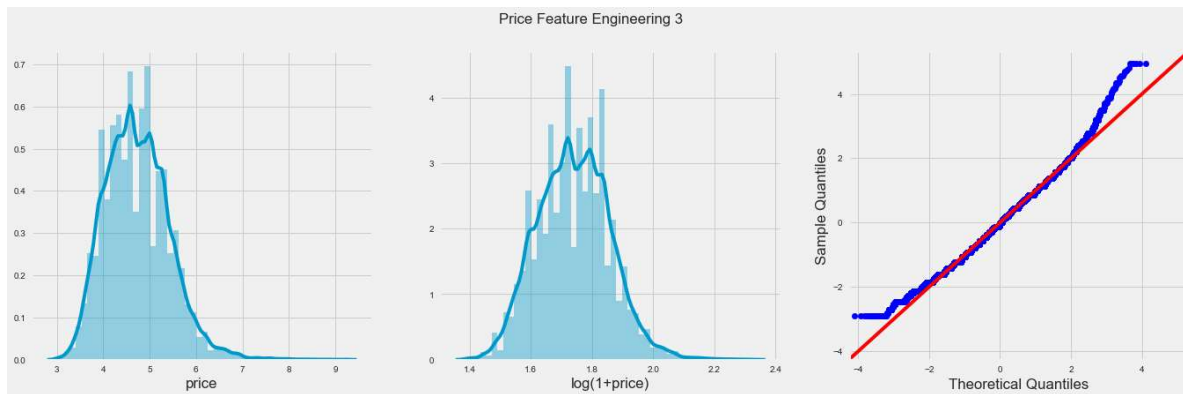
说明：然后进行对数变换，可以得到一个相对合理的价格分布数据。

In [23]:

```
1 data2['price'] = np.log1p(data2['price'])
```

In [24]:

```
1 fig, axes = plt.subplots(1,3, figsize=(21,6))
2 sns.distplot(data2['price'],color=colors[5], ax=axes[0])
3 sns.distplot(np.log1p(data2['price']),color=colors[5], ax=axes[1])
4 axes[1].set_xlabel('log(1+price)')
5 sm.qqplot(np.log1p(data2['price']),stats.norm, fit=True, line='45', ax=axes[2]);
6 fig.suptitle('Price Feature Engineering 3')
7 plt.show()
```

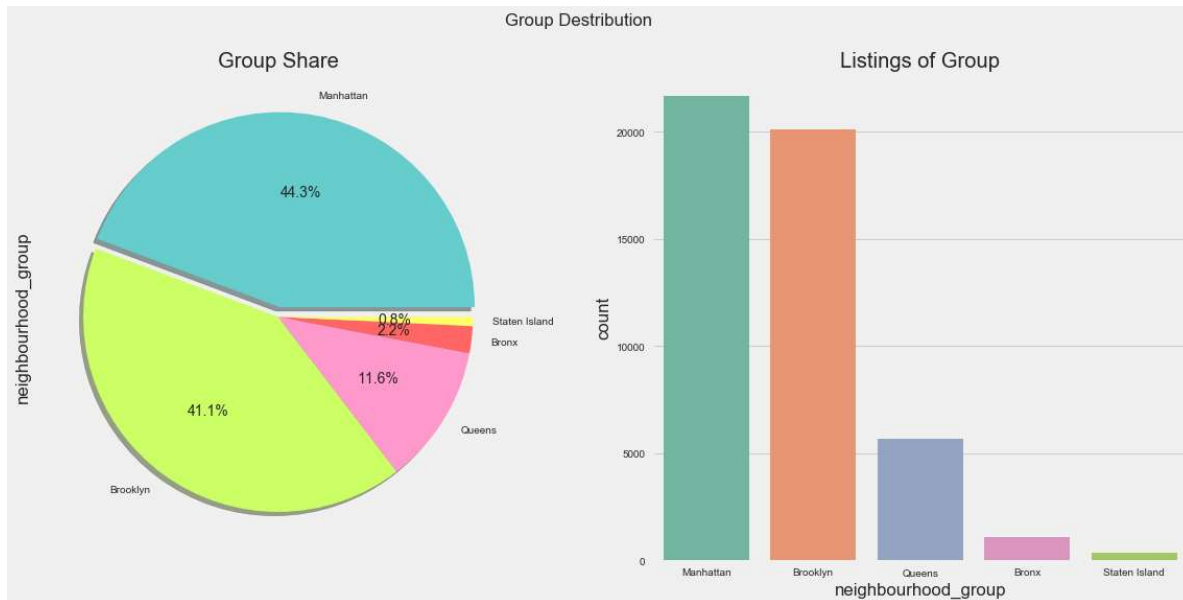


5.2 neighbourhood_group

neighbourhood_group 是美国的辖区，相当于县级区域，通过饼图与折线图可以直观看出来各个区域的订单数量情况。

In [25]:

```
1 f, ax=plt.subplots(1,2,figsize=(18,8))
2 data_origin['neighbourhood_group'].value_counts().plot.pie(explode=[0.05,0,0,0,0],colors=colors
3 ax[0].set_title('Group Share')
4 sns.countplot('neighbourhood_group',data=data_origin,ax=ax[1],order=data_origin['neighbourhood_
5 ax[1].set_title('Listings of Group')
6 f.suptitle('Group Distribution')
7 plt.show()
```



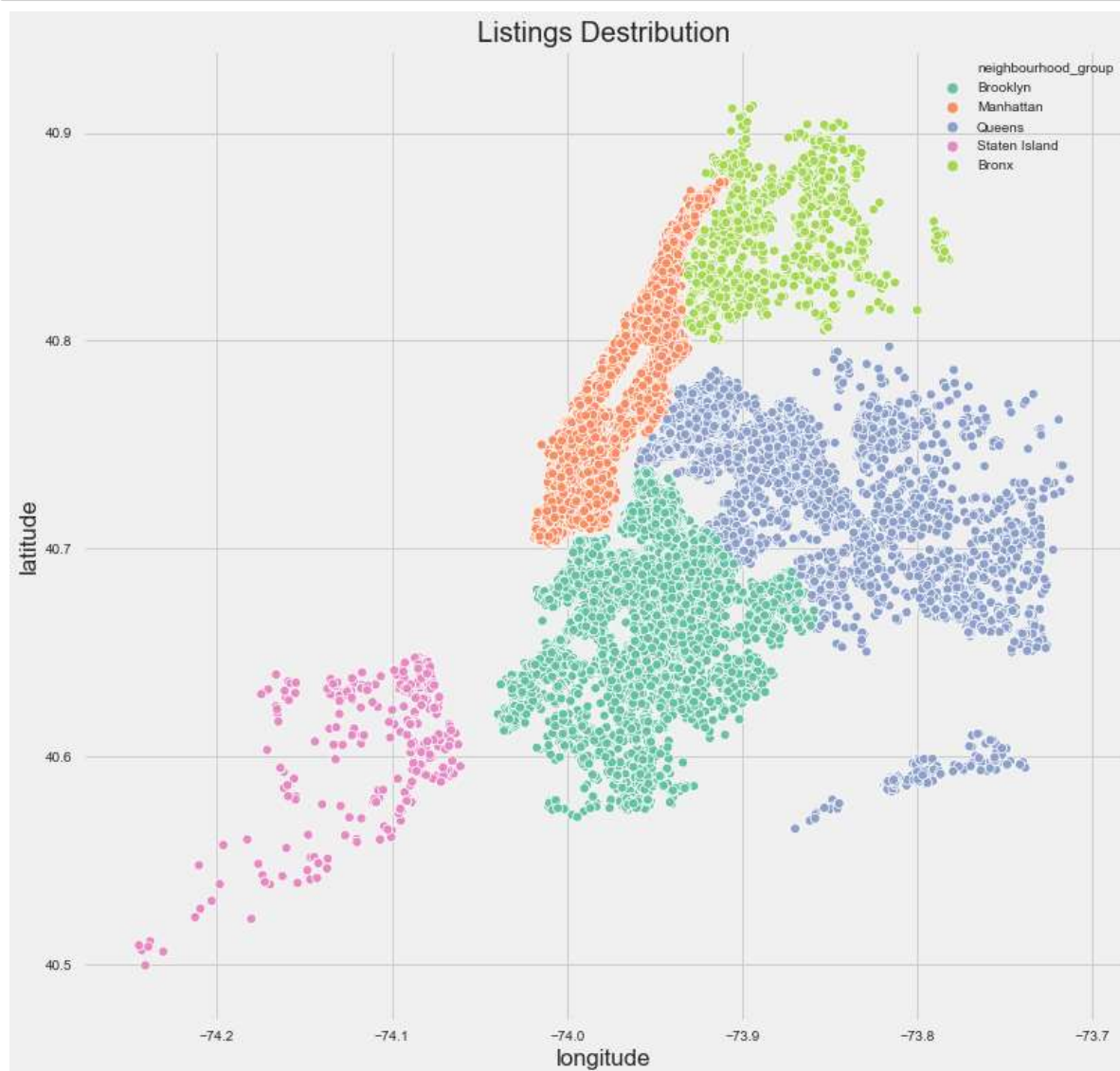
说明：可以看到在这一数据中，Manhattan、Brooklyn 与 Queens 区的订单数量占了很大的比重，有 97%，而另外的 Bronx、Staten Island 可能是比较偏远的地区，订单相对很少。

5.3 Longitude and Latitude

从经纬度信息，也可以绘制订单的分布图

In [26]:

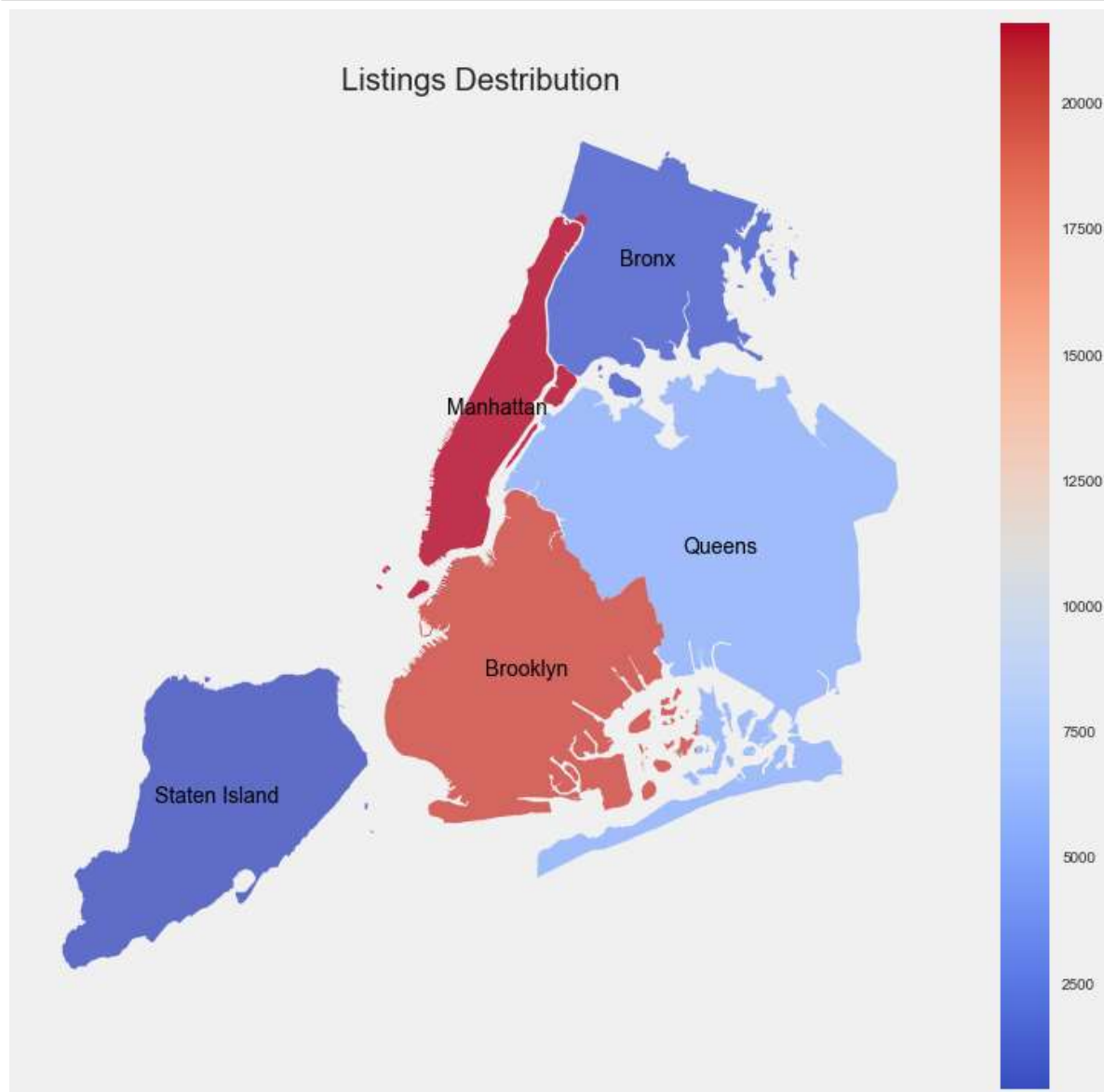
```
1 plt.style.use('fivethirtyeight')
2 plt.figure(figsize=(12, 12))
3 sns.scatterplot(data.longitude, data.latitude, hue=data.neighbourhood_group, palette="Set2")
4 plt.title("Listings Distribution")
5 plt.show()
```



说明： 由于数据较多，各区域点的分布也较为密集，通过 geopands 库可以绘制各个区域的订单热力图来进行查看。

In [27]:

```
1 borough_count = data_origin.groupby('neighbourhood_group').agg('count').reset_index()
2 nyc = gpd.read_file(gpd.datasets.get_path('nybb'))
3 nyc.head(5)
4 nyc.rename(columns={'BoroName': 'neighbourhood_group'}, inplace=True)
5 bc_geo = nyc.merge(borough_count, on='neighbourhood_group')
6 fig, ax = plt.subplots(1, 1, figsize=(12, 12))
7 bc_geo.plot(column='id', cmap='coolwarm', alpha=.8, ax=ax, legend=True)
8 bc_geo.apply(lambda x: ax.annotate(s=x.neighbourhood_group, color='black', xy=x.geometry.centroid.coords[0]))
9 plt.title("Listings Distribution")
10 plt.axis('off')
11 plt.show()
```

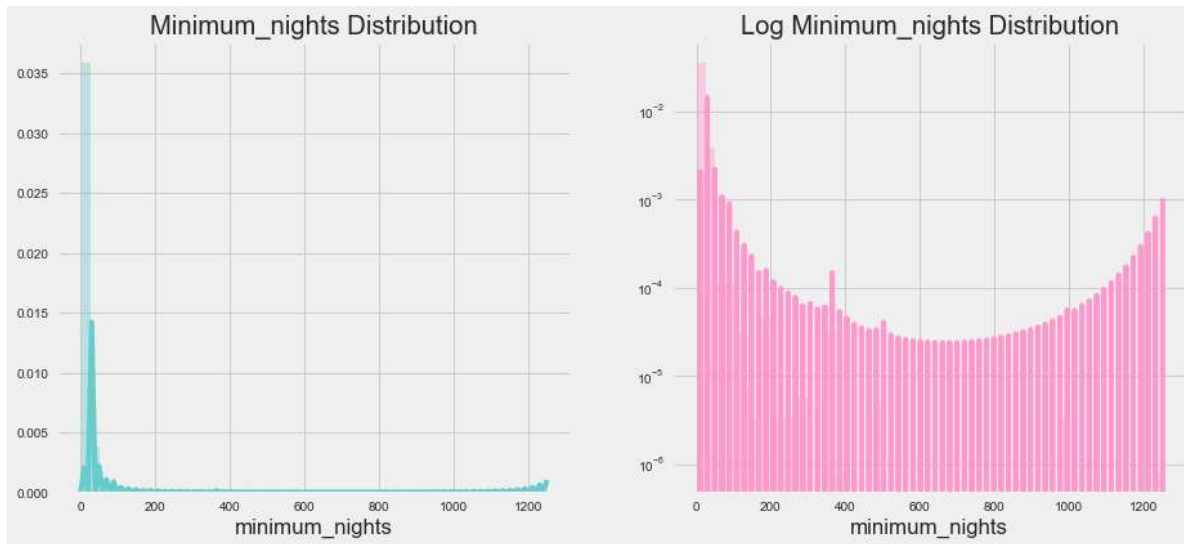


5.4 Minimum Nights

Minimum Nights 是订单的最小天数，对其进行绘制分布图进行查看。

In [28]:

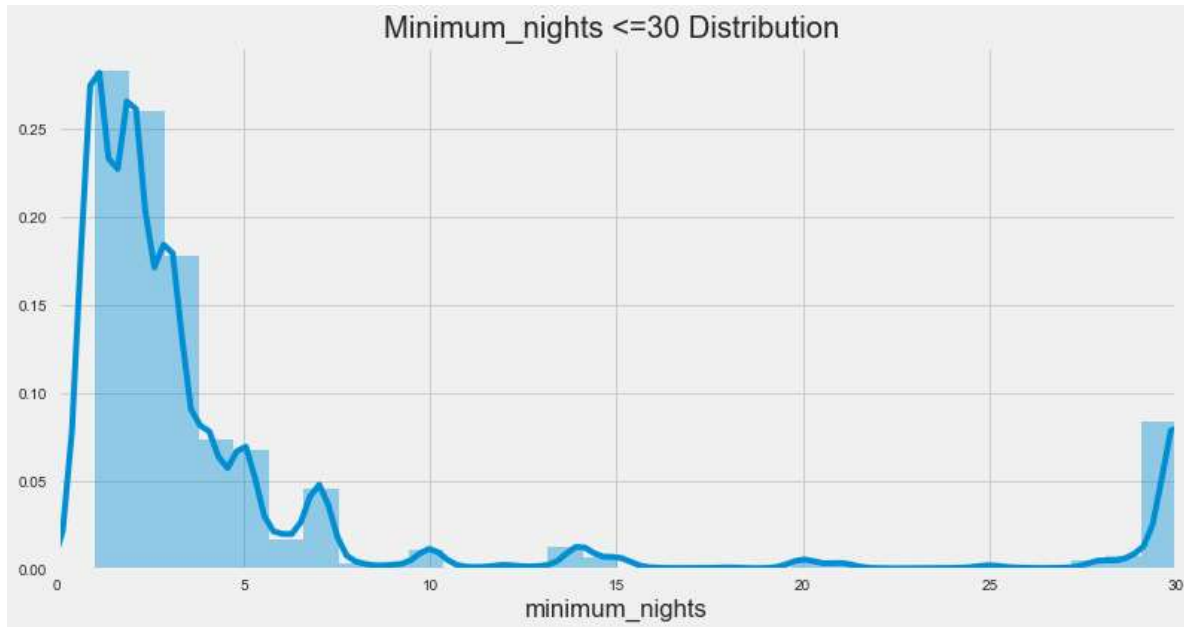
```
1 plt.style.use('fivethirtyeight')
2 plt.figure(figsize=(14,6))
3 plt.subplot(1,2,1)
4 sns.distplot(data_origin.minimum_nights,color=colors[0])
5 plt.title('Minimum_nights Distribution')
6 plt.subplot(1,2,2)
7
8 sns.distplot(data_origin.minimum_nights,color=colors[2]).set_yscale('log')
9 plt.title('Log Minimum_nights Distribution')
10 plt.show()
```



说明：可以看到 Minimum Nights 的分布也较为极端，同样进行对数转换后的数据分布显得更为合理，需要进行特征工程。

In [29]:

```
1 plt.style.use('fivethirtyeight')
2 plt.figure(figsize=(12,6))
3 sns.distplot(data_origin[(data_origin['minimum_nights'] <= 30) & (data_origin['minimum_nights']
4 plt.xlim(0,30)
5 plt.title('Minimum_nights <=30 Distribution ')
6 plt.show()
```



说明：通过观察小于 30 天的订单情况，可以发现用户大多会预定 1-5 天。

In [30]:

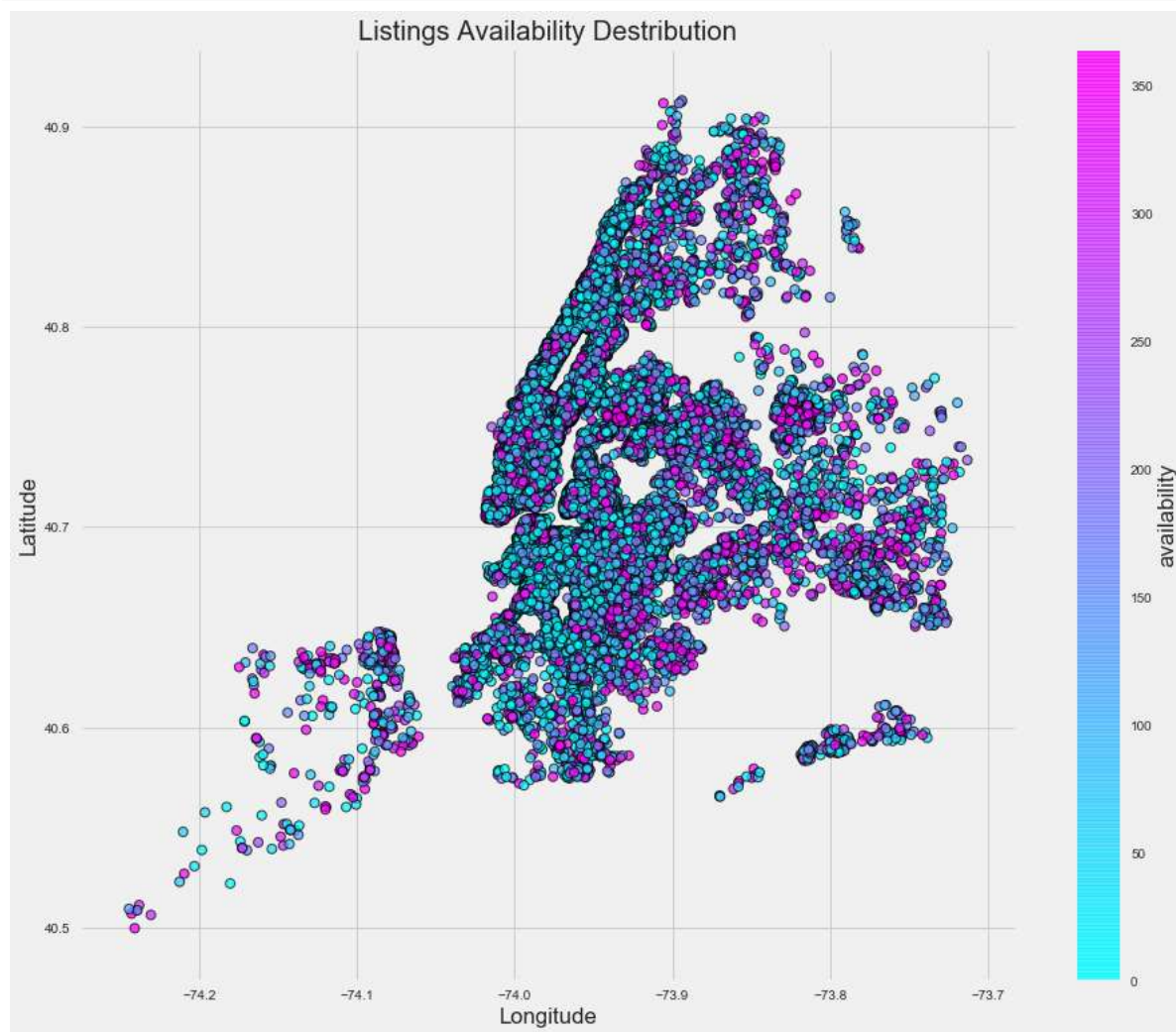
```
1 data2['minimum_nights'] = np.log1p(data_origin['minimum_nights'])
```

5.5 Availability

订单预定时的可用天数也是需要进行区域可视化的属性。

In [31]:

```
1 plt.figure(figsize=(14, 12))
2 plt.scatter(data_origin.longitude, data_origin.latitude, c=data_origin.availability_365, cmap=
3 cbar = plt.colorbar()
4 plt.title("Listings Availability Distribution")
5 plt.xlabel("Longitude")
6 plt.ylabel("Latitude")
7 cbar.set_label('availability')
8 plt.show()
```



说明：从可用天数的区域分布也可以看出，处于纽约市中心区域的房子可用天数往往较低，颜色多为 colorbar 的蓝色，而处于边远区域的房子更倾向于为紫色，即可用天数较多。

In [32]:

```
1 fig, axes = plt.subplots(1,1,figsize=(18.5, 6))
2 sns.distplot(data_origin['availability_365'], color="#66ccff", ax=axes)
3 axes.set_xlabel('availability_365')
4 plt.title("Listings Availability <=365 Destribution")
5 axes.set_xlim(0,365)
6 plt.show()
```



说明：从其数量分布来看，可用天数明显倾向于两端分布，大部分的房子都只在几天内可用，也有一部分房子可以超过 365 天。因此可以尝试对其进行按照可用天数分类。

In [33]:

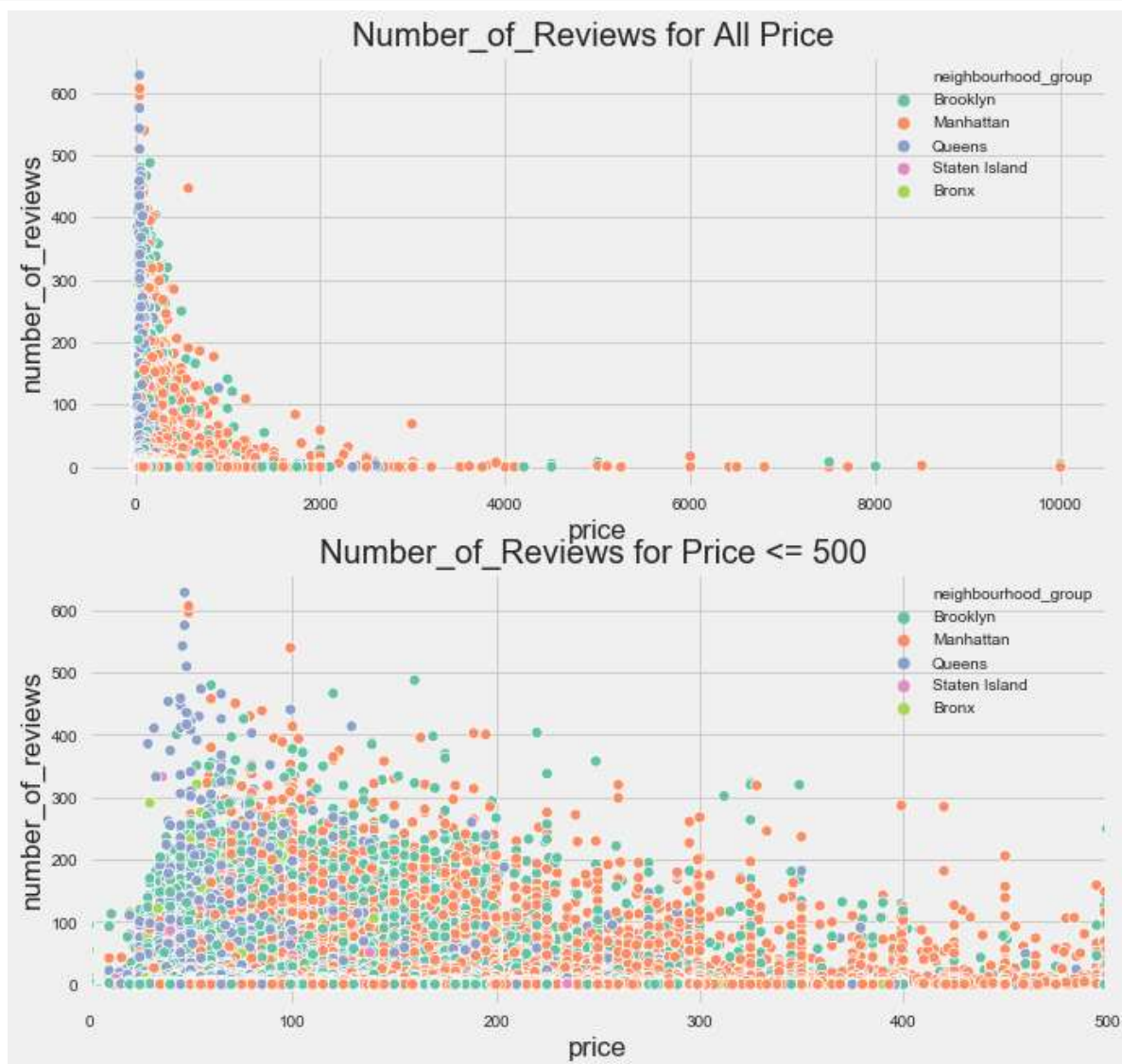
```
1 data2['all_year_avail']=data2.apply(lambda x: 1 if x.availability_365>350 else 0, axis=1)
2 data2['low_avail']=data2.apply(lambda x: 1 if x.availability_365<14 else 0, axis=1)
```

5.6 Review

与 Review 相关的属性有 `number_of_reviews` , `last_review` , `reviews_per_month` 。其中 `number_of_reviews` 和 `reviews_per_month` 是具有数值意义的。

In [34]:

```
1 from matplotlib.pyplot import MultipleLocator
2 fig, axes = plt.subplots(2,1, figsize=(10,10))
3 sns.scatterplot(x= data_origin['price'], y=data_origin['number_of_reviews'],hue =data_origin.ne
4 sns.scatterplot(x= data_origin['price'], y=data_origin['number_of_reviews'],hue =data_origin.ne
5 plt.xlim(0,500)
6 axes[0].set_title('Number_of_Reviews for All Price ')
7 axes[1].set_title('Number_of_Reviews for Price <= 500 ')
8 plt.show()
```



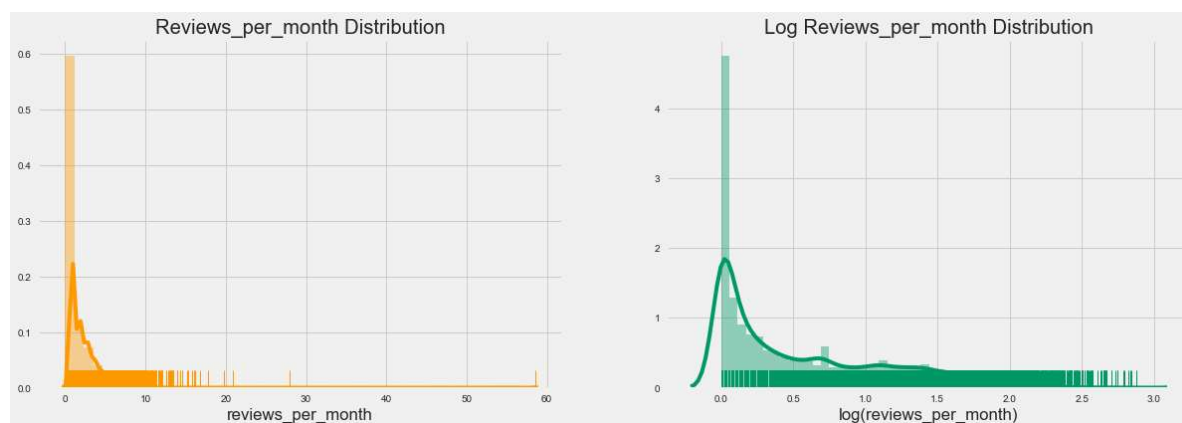
说明：从价格与评论数量的散点图可以发现价格较低的房子更倾向于有更多的评论，价格约为 50 dollar 的房子拥有数量最多的评论。而 2000 dollar 及以上的房子几乎没有评论，所以评论数量与价格有着高度的相关性。

In [35]:

```
1 fig, axes = plt.subplots(1,2,figsize=(18.5, 6))
2 sns.distplot(data_origin['reviews_per_month'], rug=True, kde=True, color="#FF9900", ax=axes[0])
3 sns.distplot(np.log1p(data_origin[data_origin['reviews_per_month'] < 17.5]['reviews_per_month']),
4 axes[1].set_xlabel('log(reviews_per_month)')
5 axes[0].set_title('Reviews_per_month Distribution')
6 axes[1].set_title('Log Reviews_per_month Distribution')
```

Out[35]:

Text(0.5, 1.0, 'Log Reviews_per_month Distribution')



说明：对于评论这一属性，即使用对数变换来改变数据的分布，也难以使其改变其在低价位极密分布的情况，因此便不再详细进行区间划分。但是可以另外加一列没有评论的数据，可判断 no_reviews 是否为 0 进行添加操作。

In [36]:

```
1 data2['no_reviews'] = data2.apply(lambda x: 1 if x.reviews_per_month==0 else 0, axis=1)
```

In [37]:

```

1 plt.style.use('seaborn')
2 plt.subplots(figsize=(12, 12))
3 text = " ".join(str(each) for each in data_origin.name)
4 mask=plt.imread('14.jpeg')
5 wordcloud = WordCloud(
6     max_words=100,
7     background_color='white',
8     width=3000,
9     mask=mask,
10    height=3000,
11    random_state=30,
12    colormap='Set2'
13 ).generate(text)
14 plt.axis("off")
15 plt.imshow(wordcloud)
16 plt.title("Word Cloud for Listing Name")
17 plt.savefig('neighbourhood.png')
18 plt.show()

```



说明：用订单的 name 属性绘制了词云，除了地名以外，还可以看到其他的一些形容词，如 private、beautiful、cozy、spacious 等等。可见很多房东都会用这些描述性的词语来吸引房客选择他们的出租方案。

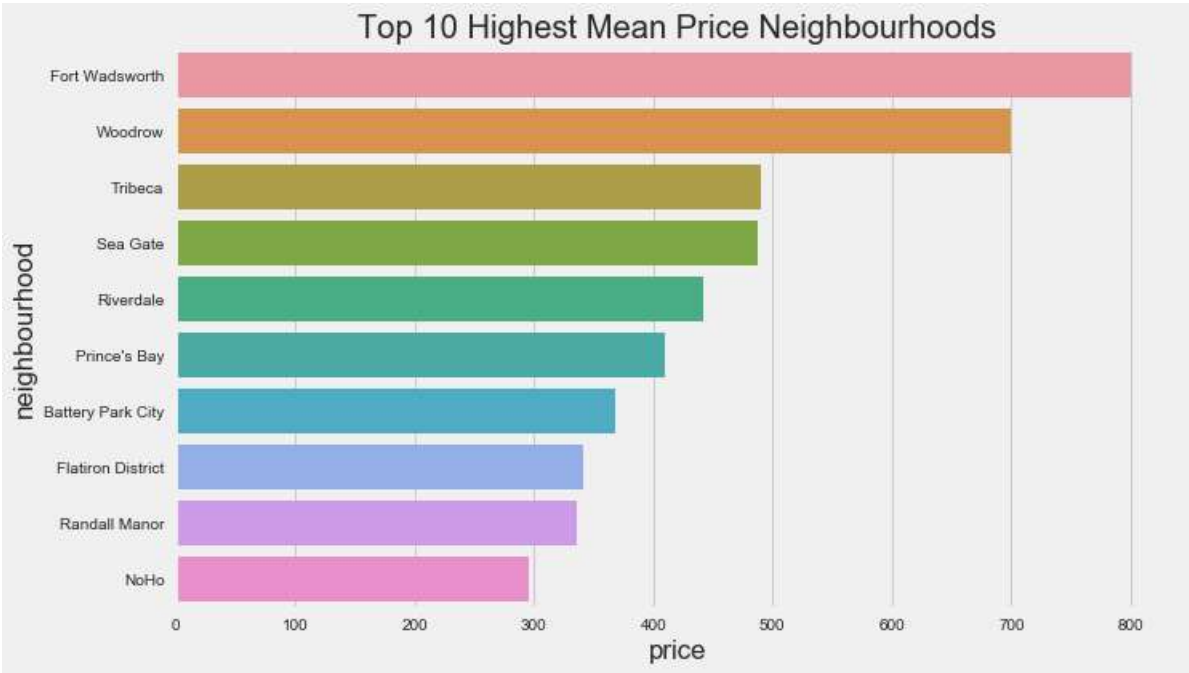
5.8 Neighbourhood

Neighbourhood 是美国的街区单位，在这一数据集中有许多种类的街区。

In [38]:

```
1 plt.style.use('fivethirtyeight')
2 print ("Total Neighbourhoods: ", len(np.unique(data_origin.neighbourhood)))
3 df_top_prices_by_neighbourhood = data.groupby('neighbourhood').agg({'price': 'mean'}).sort_valu
4 plt.figure(figsize=(10,6))
5 sns.barplot(y="neighbourhood", x="price", data=df_top_prices_by_neighbourhood.head(10))
6 plt.title('Top 10 Highest Mean Price Neighbourhoods')
7 plt.show()
```

Total Neighbourhoods: 221



说明：从平均房价最高的 Neighbourhood 前十名排行可以看到，较高的房租大部分在 300-500dollar，相对最贵的地区应为 Fort Wadsworth，平均价格高达800dollar。

In [39]:

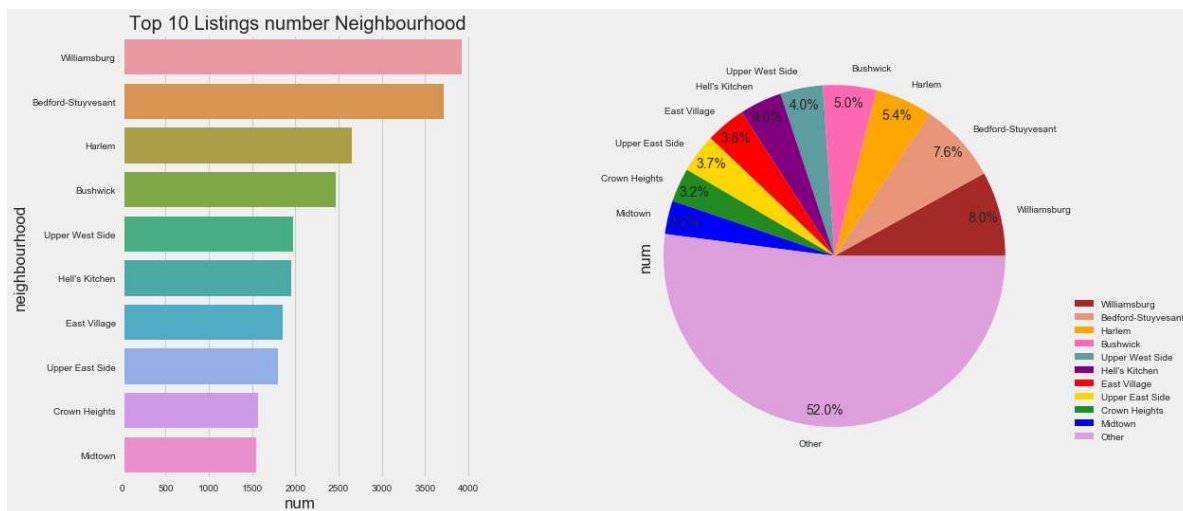
```
1 plt.style.use('fivethirtyeight')
2 plt.figure(figsize=(12,6))
3 sns.barplot(y="neighbourhood", x="price", data=data_origin.nlargest(10,['price']))
4 plt.title('Top 10 Highest Max Price Neighbourhoods')
5 plt.show()
```



说明：对比平均房价和最高房价的前十 Neighbourhood 榜单，可以发现有着明显的不同，Astoria、Greenpoint、Upper West Side、East Harlem、Lower East Side、这些地区的最高房价都达到了10000\$，说明这些地区存在着较为豪华奢侈的房型。

In [40]:

```
1 fig, axes=plt.subplots(1,2,figsize=(15,8))
2
3 nei_number=data_origin['neighbourhood'].value_counts().sort_values(ascending=False)[:10]
4 l1=[]
5 for i in range(0,10):
6     l1.append(int(nei_number.iloc[i]))
7 l2=list(nei_number.index)#group
8 nei=pd.DataFrame(columns=['neighbourhood'],data=l2)
9 nei['num']=l1 #num
10 sns.barplot(y="neighbourhood", x="num", data=nei.head(10),ax=axes[0])
11 axes[0].set_title("Top 10 Listings number Neighbourhood ")
12
13 all_num=data_origin['neighbourhood'].value_counts()
14 l1.append(all_num.agg('sum')-all_num[:10].agg('sum'))
15 l2.append('Other')
16 pie_data=pd.DataFrame({'neighbourhood':l2,"num":l1})
17 clr1=('brown','darksalmon','orange','hotpink','cadetblue','purple','red','gold','forestgreen','',
18 qx = pie_data.plot(kind='pie', y='num', labels=l2,colors=clr1,autopct='%1.1f%%', pctdistance=0
19 plt.legend(loc=0, bbox_to_anchor=(1.15,0.4))
20 plt.subplots_adjust(wspace =0.5, hspace =0)
21 plt.show()
```

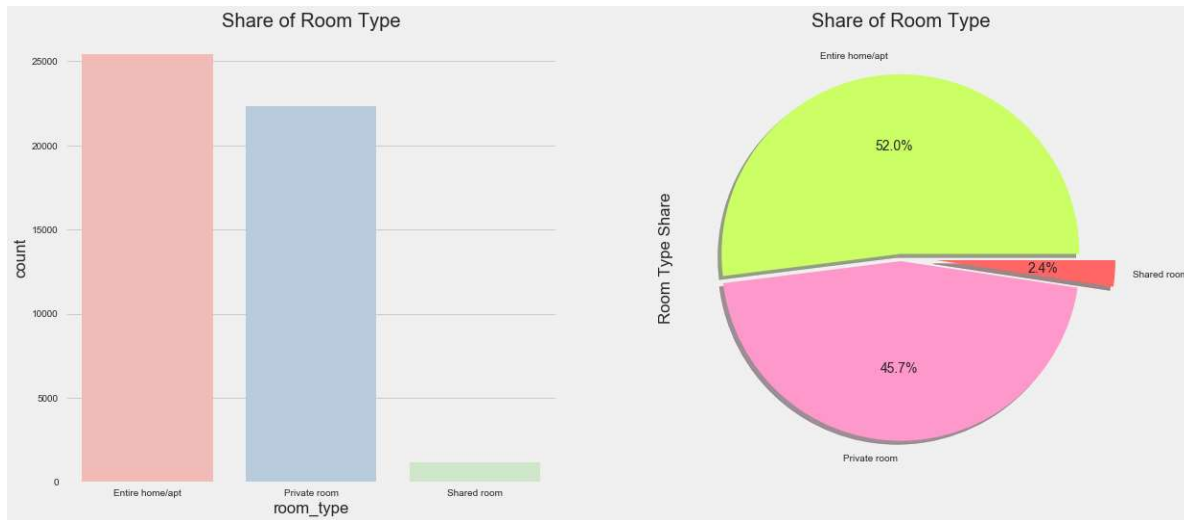


说明：从 Neighbourhood 订单的柱状图和饼图可以观察到，订单数量最多的前十名 Neighbourhood 的订单总和已接近所有订单的 50%，这些地区贡献了大部分房源，Williamsburg 与 Bedford-Stuyvesant 的订单数量都接近 4000，数量众多。

5.9 room_type

In [41]:

```
1 f, ax=plt.subplots(1,2,figsize=(18,8))
2 sns.countplot('room_type', data=data_origin, ax=ax[0], palette="Pastell1", order=data_origin['room_type'].value_counts().index)
3 ax[0].set_title('Share of Room Type')
4 data_origin['room_type'].value_counts().plot.pie(colors=colors[1:], explode=[0.02, 0.02, 0.2], autopct='%1.1f%%')
5 ax[1].set_title('Share of Room Type')
6 ax[1].set_ylabel('Room Type Share')
7
8 plt.show()
```

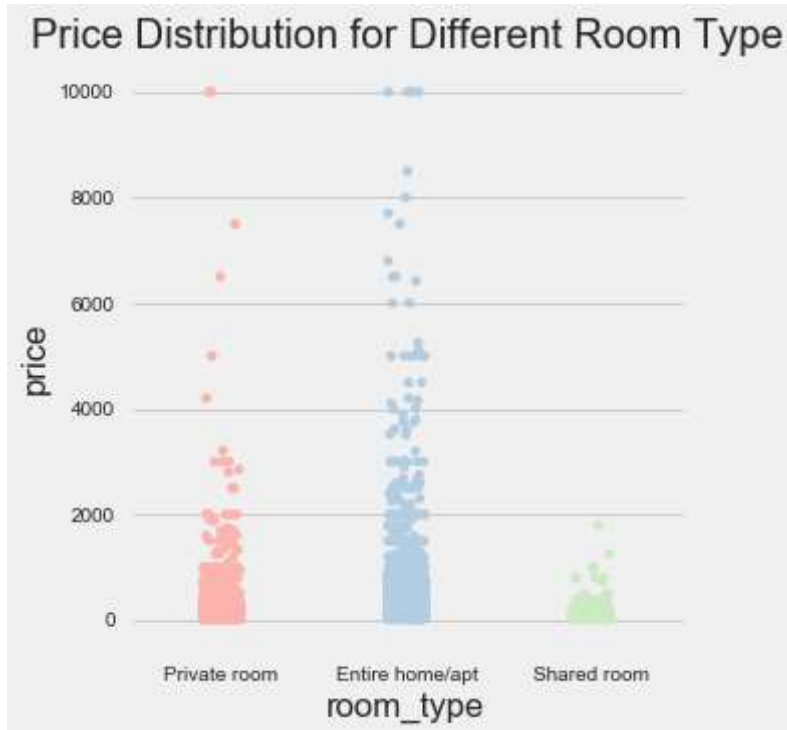


说明：从 room_type 的分布情况可以发现，Entire home/apt 和 Private room 占了所有订单的大部分，分别为 52.0% 和 45.7%，而只有极少数人选择了 Shared room，可能是租客比较注重个人隐私空间的原因使得房型呈现这样的分布。

In [42]:

```
1 plt.figure(figsize=(10,6))
2 sns.catplot(x="room_type", y="price", data=data_origin,palette="Pastell");
3 plt.title('Price Distribution for Different Room Type')
4 plt.show()
```

<Figure size 720x432 with 0 Axes>



说明：从不同房型的价格分布来看，Entire home/apt 的价格分布区间更广，有更多个价位选择，而 Shared room 的价格往往较低，最高不过2000\$。

相关系数

最终经过以上属性分析和特征工程的处理数据之间的相关系数如下

In [43]:

```
1 data2.corr().style.background_gradient(cmap='magma')
```

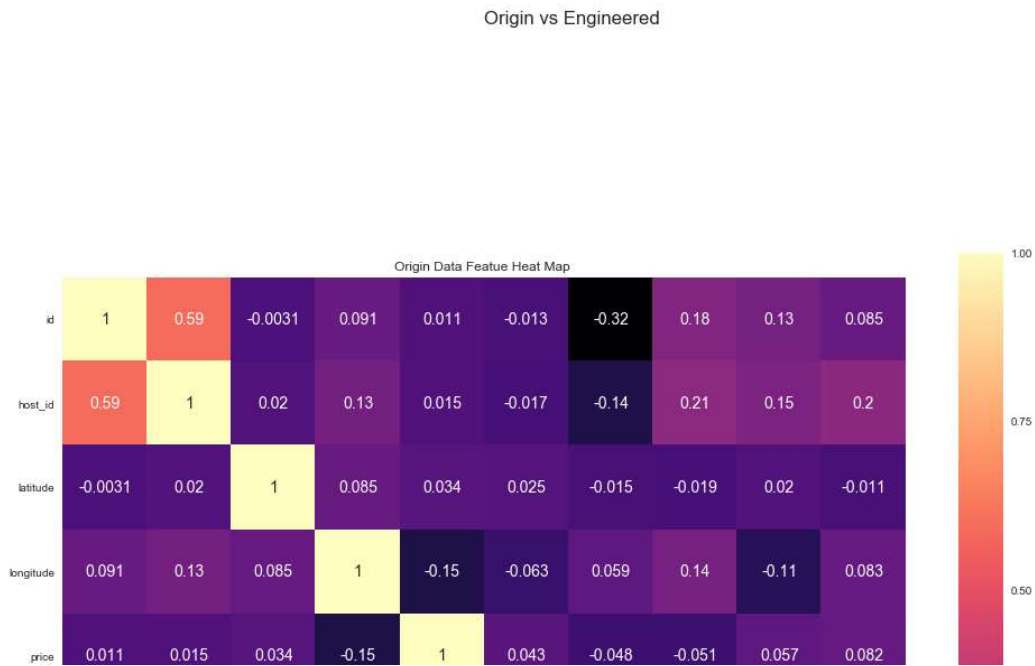
Out[43]:

	id	host_id	latitude	longitude	price	min
id	1	0.588312	-0.00292689	0.0908054	-0.0143785	
host_id	0.588312	1	0.0203715	0.126622	-0.0160146	
latitude	-0.00292689	0.0203715	1	0.084891	0.0793807	
longitude	0.0908054	0.126622	0.084891	1	-0.32668	
price	-0.0143785	-0.0160146	0.0793807	-0.32668	1	
minimum_nights	-0.0254717	-0.0640644	0.0331589	-0.126294	0.0641268	
number_of_reviews	-0.319826	-0.140175	-0.0153507	0.0589992	-0.0434111	
reviews_per_month	0.180324	0.209848	-0.018585	0.138659	-0.0609899	
calculated_host_listings_count	0.133375	0.155095	0.0195276	-0.114765	0.132951	
availability_365	0.0852709	0.203292	-0.0110266	0.0820434	0.0995794	
all_year_avail	0.0341901	0.0932927	-0.00371615	0.0569728	0.0594624	
low_avail	-0.193388	-0.239786	0.0192163	-0.109211	-0.0791097	
no_reviews	0.164906	0.0846962	0.0291765	-0.0434331	0.0855509	

运用热力图，我们可以直观比较其与原始数据的差异

In [44]:

```
1 plt.style.use('seaborn')
2 f, ax=plt.subplots(2,1,figsize=(15,30))
3
4 sns.heatmap(data_origin.corr(), vmax=1, ax=ax[0], square=True, annot=True, cmap='magma')
5
6 sns.heatmap(data2.corr(), vmax=1, ax=ax[1], square=True, annot=True, cmap='magma')
7 ax[0].set_title('Origin Data Feature Heat Map ')
8 ax[1].set_title('Engineered Feature Heat Map')
9 f.suptitle('Origin vs Engineered')
10 plt.show()
11
```



说明：可以看到数据多出了几小类别，整体也显得“更亮”了，这是由上面的属性分类和调整数据分布达到的效果。

6、使用特征工程后的数据进行模型建立

同之前的操作，drop 无关属性后，对标称数据进行哑变量或连续化。

In [45]:

```
1 data2=data2.drop(columns=['host_id', 'id', 'name', 'host_name'], axis=1)
2 data2['last_review'] = data2['last_review'].apply(lambda x:x.toordinal() - earliest.toordinal())
3 data2_ob=data2.select_dtypes(include=['object'])
4 print(data2_ob.columns)
5 data2_ob_oh=pd.get_dummies(data2_ob)
6 data2_ob_oh.head(5)
```

Index(['neighbourhood_group', 'neighbourhood', 'room_type'], dtype='object')

Out[45]:

	neighbourhood_group_Bronx	neighbourhood_group_Brooklyn	neighbourhood_group_Manhattan
0	0	1	0
1	0	0	1
2	0	0	0
3	0	1	0
4	0	0	1

5 rows × 229 columns

In [46]:

```
1 data2_num= data2.select_dtypes(exclude=['object'])
2 y = data2_num.loc[:, 'price']
3 data2_num.drop(columns=['price'], axis=1, inplace=True)
4 data2_pre = pd.concat([data2_num, data2_ob_oh, y], axis=1)
5 print(data2_pre.shape)
6 data2_pre.columns.tolist()
```

(48841, 241)

In [47]:

```
1 X=data2_pre.iloc[:, :-1]
2 y = data2_pre.iloc[:, -1]
3 X_train,X_test,y_train,y_test = train_test_split(X,y, test_size=0.2, random_state=42)
4 print(f'The shape of Xtrain:{X_train.shape}')
5 print(f'The shape of Xtest:{X_test.shape}')
6 print(f'The shape of ytrain:{y_train.shape}')
7 print(f'The shape of ytest:{y_test.shape}')
8 scaler = RobustScaler()
9 X_train = scaler.fit_transform(X_train)
10 X_test = scaler.fit_transform(X_test)
11
```

The shape of Xtrain:(39072, 240)

The shape of Xtest:(9769, 240)

The shape of ytrain:(39072,)

The shape of ytest:(9769,)

In [48]:

```
1 def r2_score(model):
2     return cross_val_score(model,X_train,y_train, cv=5)
3 def rmse_score(model):
4     return cross_val_score(model,X_train,y_train, scoring='neg_mean_squared_error', cv=5)
5
6 Model=[LinearRegression, Ridge, Lasso, HuberRegressor, RandomForestRegressor, XGBRegressor]
7 lname2=[]
8 lr22=[]
9 lrmse2=[]
10 for model in Model:
11     if model == XGBRegressor:
12         m=XGBRegressor(objective='reg:squarederror')
13     else:
14         m=model()
15     cv_r2=r2_score(m)
16     cv_rmse_score = rmse_score(m)
17     print('The mean R^2 of {} is: {:.3e}'.format(model.__name__, cv_r2.mean()))
18     print('The mean Mse of {} is: {:.3e}'.format(model.__name__, -cv_rmse_score.mean()))
19     lname2.append(model.__name__)
20     lr22.append(cv_r2.mean())
21     lrmse2.append(-cv_rmse_score.mean())
```

The mean R² of LinearRegression is:-2.334e+13

The mean Mse of LinearRegression is: 1.107e+13

The mean R² of Ridge is:5.601e-01

The mean Mse of Ridge is: 2.106e-01

The mean R² of Lasso is:1.571e-02

The mean Mse of Lasso is: 4.713e-01

The mean R² of HuberRegressor is:5.339e-01

The mean Mse of HuberRegressor is: 2.231e-01

The mean R² of RandomForestRegressor is:5.615e-01

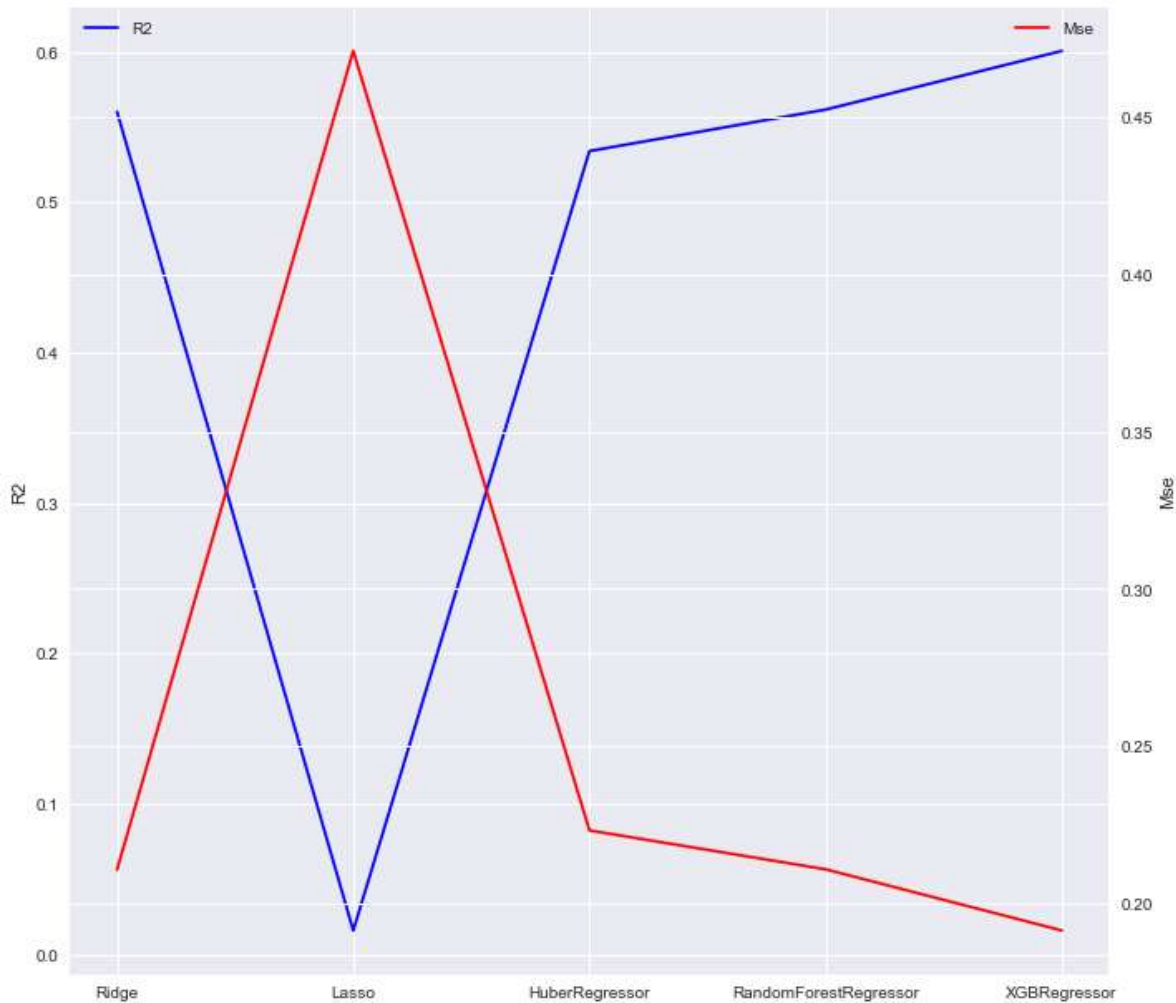
The mean Mse of RandomForestRegressor is: 2.108e-01

The mean R² of XGBRegressor is:6.005e-01

The mean Mse of XGBRegressor is: 1.913e-01

In [49]:

```
1 fig = plt.figure(figsize=(10,10))
2 ax1 = fig.add_subplot(111)
3 ax1.plot(lname2[1:], lr22[1:], 'b', label="R2");
4 ax1.legend(loc=2)
5 ax1.set_ylabel('R2');
6 ax2 = ax1.twinx() # this is the important function
7 ax2.plot(lname2[1:], lrmse2[1:], 'r', label = "Mse")
8 ax2.legend(loc=1)
9 # ax2.set_xlim([0, np.e]);
10 ax2.set_ylabel('Mse');
11 plt.show()
```



说明：再次对几个模型进行验证对比，依然是 XGBRegressor 的效果较好，且 R^2 达到了 0.6，MSE 低至 0.2 可以对其进行绘制决策树和特征重要度。

In [50]:

```
1 model = XGBRegressor(objective='reg:squarederror')
2 model.fit(X_train, y_train)
3 model.save_model('xgbr.model')
4 feature=data2_pre.columns
5 feature
```

Out[50]:

```
Index(['latitude', 'longitude', 'minimum_nights', 'number_of_reviews',
      'last_review', 'reviews_per_month', 'calculated_host_listings_count',
      'availability_365', 'all_year_avail', 'low_avail',
      ...
      'neighbourhood_Willowbrook', 'neighbourhood_Windsor Terrace',
      'neighbourhood_Woodhaven', 'neighbourhood_Woodlawn',
      'neighbourhood_Woodrow', 'neighbourhood_Woodside',
      'room_type_Entire home/apt', 'room_type_Private room',
      'room_type_Shared room', 'price'],
      dtype='object', length=241)
```

In [51]:

```
1 feature=data2_pre.columns
2 feature
```

Out[51]:

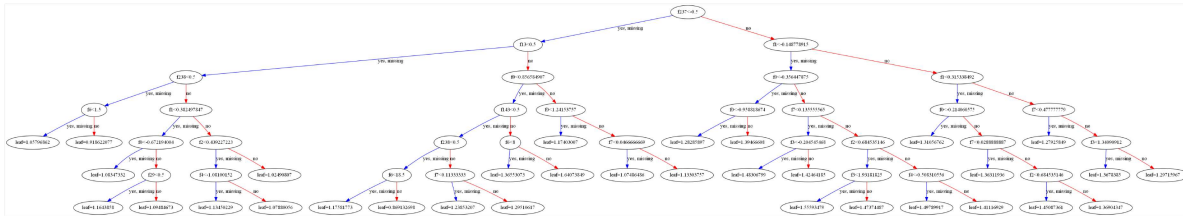
```
Index(['latitude', 'longitude', 'minimum_nights', 'number_of_reviews',
      'last_review', 'reviews_per_month', 'calculated_host_listings_count',
      'availability_365', 'all_year_avail', 'low_avail',
      ...
      'neighbourhood_Willowbrook', 'neighbourhood_Windsor Terrace',
      'neighbourhood_Woodhaven', 'neighbourhood_Woodlawn',
      'neighbourhood_Woodrow', 'neighbourhood_Woodside',
      'room_type_Entire home/apt', 'room_type_Private room',
      'room_type_Shared room', 'price'],
      dtype='object', length=241)
```

In [52]:

```
1 # def ceate_feature_map(features):
2 #     outfile = open('xgbl.fmap', 'w')
3 #     i = 0
4 #     for feat in features:
5 #         outfile.write('{0}\t{1}\tq\n'.format(i, feat))
6 #         i = i + 1
7 #     outfile.close()
8 # ceate_feature_map(feature)#特征名列表
```

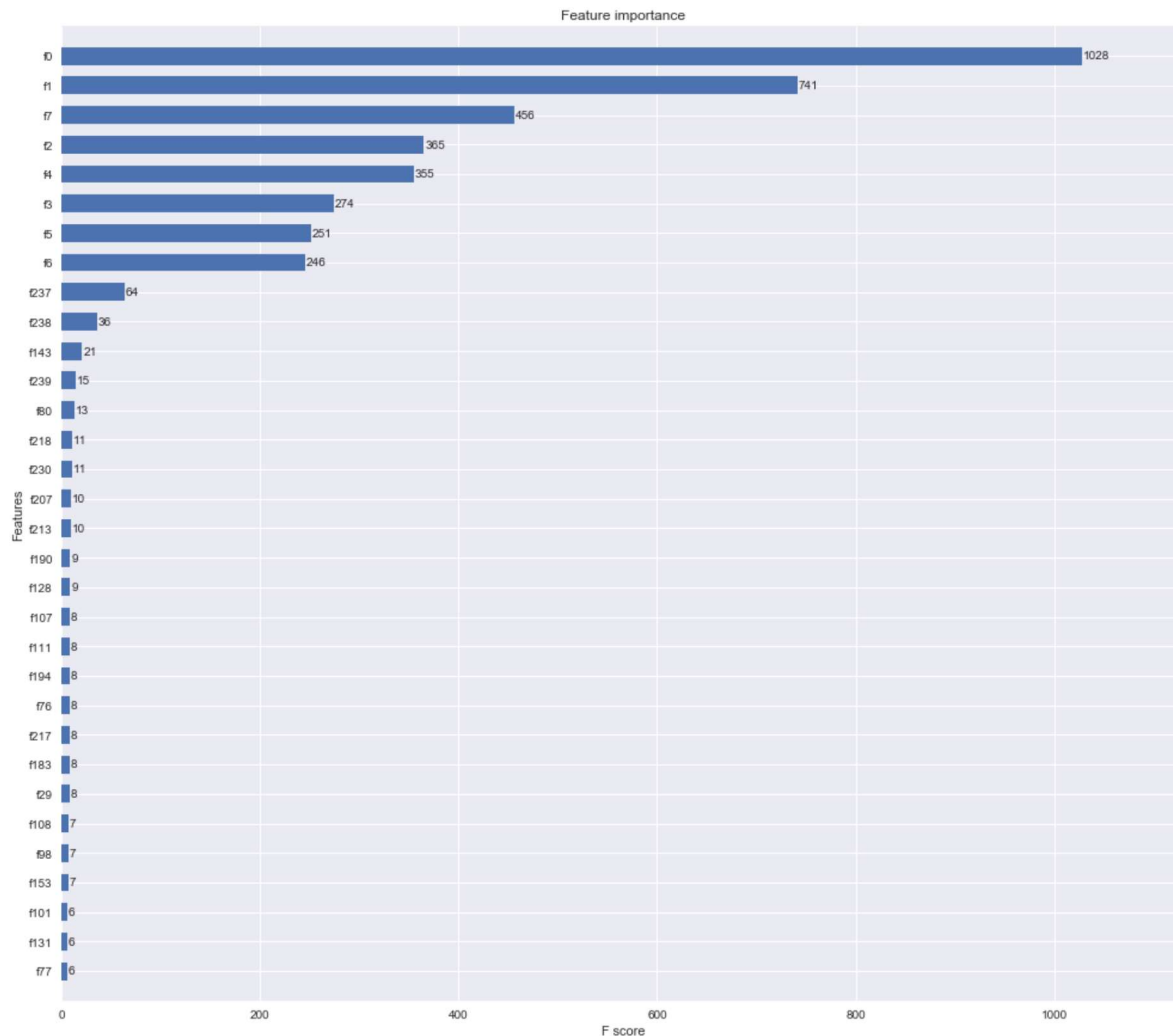
In [53]:

```
1 %matplotlib inline
2 f,ax=plt.subplots(1,1,figsize=(15,10))
3 xgb.plot_tree(model, num_trees=0,ax=ax)
4 fig = plt.gcf()
5 fig.set_size_inches(150, 100)
6 plt.savefig('tree.png')
7 plt.show()
```



In [54]:

```
1 fig,ax = plt.subplots(figsize=(15,15))
2 xgb.plot_importance(model,
3                     height=0.6,
4                     ax=ax, importance_type='weight',
5                     max_num_features=32
6                     )
7 plt.show()
```



说明: 比较重要的为第 0、1、7、2、4、3、5、6 个特征。

In [55]:

```
1 fi=[0,1,7,2,4,3,5,6]
2 fl=list(feature)
3 for i in range(len(fi)):
4     fi[i]=fl[i]
5 fi
```

Out[55]:

```
['latitude',
 'longitude',
 'minimum_nights',
 'number_of_reviews',
 'last_review',
 'reviews_per_month',
 'calculated_host_listings_count',
 'availability_365']
```

说明：经纬度成为了比较重要的预测特征，可以说明房价的区域性变化是比较明显的，在可视过程中也可以印证这一点。

7、利用 Featuretools 自动特征工程

为了避免自动特征过程中产生较多的 nan 和 inf，补充缺失值时考虑填入 1，只将处理后的数值性属性进行自动特征工程，再与哑变量特征合并形成数据集。

In [56]:

```
1 import featuretools as ft
2
3 data3=pd.read_csv('AB_NYC_2019.csv')
4 data3=data3.drop(columns=['host_id', 'id','name','host_name'], axis=1)
5 data3['reviews_per_month'] = data3['reviews_per_month'].fillna(1)
6 data3['last_review'] = pd.to_datetime(data3['last_review'], infer_datetime_format=True)
7 earliest = min(data3['last_review'])
8 data3['last_review'] = data3['last_review'].fillna(earliest)
9 data3['last_review'] = data3['last_review'].apply(lambda x:x.toordinal() - earliest.toordinal())
10 data3_ob=data3.select_dtypes(include=['object'])
11 data3_ob_ob=pd.get_dummies(data3_ob)
```

get_dummies 生成哑变量部分，数值型导入 featuretools 的 entity。

In [57]:

```
1 data3_num= data3. select_dtypes(exclude=['object'])
2 y = data3_num.loc[:, 'price']
3 data3_num.drop(columns=['price'], axis=1, inplace=True)
4 es = ft.EntitySet(id='ftdata')
5 es.entity_from_dataframe(entity_id='data3',
6                           dataframe=data3_num,
7                           index='index',
8                           )
9
```

```
2021-01-09 16:28:10,736 featuretools.entityset - WARNING      index index not found in
dataframe, creating new integer column
```

Out[57]:

```
Entityset: ftdata
Entities:
  data3 [Rows: 48895, Columns: 9]
Relationships:
  No relationships
```

In [58]:

```
1 es['data3'].variables
2 es.plot()
```

Out[58]:

```
<graphviz.dot.Digraph at 0x18faed59648>
```

数值型特征已导入，这里用加减乘除对他们进行 dfs: 自动特征生成。

In [59]:

```
1 trans_primitives=['add_numeric', ft.primitives.SubtractNumeric(commutative=False), 'multiply_n
2 feature_matrix, feature_names = ft.dfs(entityset=es,
3     target_entity='data3',
4     max_depth=1,
5     verbose=1,
6     trans_primitives=trans_primitives
7 )
```

Built 176 features

[illegible]

得到的特征容易出现 inf 和 nan，这里特征的语义已经不易进行特别考虑来处理缺失值，直接 drop。

In [60]:

```
1 feature_matrix=feature_matrix.replace([np.inf, -np.inf], np.nan)
2 feature_matrix=feature_matrix.dropna(axis=0,how='any')
3 feature_matrix.isnull().sum()
4 print(feature_matrix.shape)
```

(26155, 176)

再次生成数据集进行测试

In [61]:

```
1 data3_pre = pd.concat([feature_matrix, data3_ob_oh, y], axis=1, join='inner')
```

In [62]:

```
1 X=data3_pre.iloc[:, :-1]
2 y = data3_pre.iloc[:, -1]
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
4 print(f'The shape of Xtrain:{X_train.shape}')
5 print(f'The shape of Xtest:{X_test.shape}')
6 print(f'The shape of ytrain:{y_train.shape}')
7 print(f'The shape of ytest:{y_test.shape}')
8 scaler = RobustScaler()
9 X_train = scaler.fit_transform(X_train)
10 X_test = scaler.fit_transform(X_test)
```

The shape of Xtrain:(20924, 405)

The shape of Xtest:(5231, 405)

The shape of ytrain:(20924,)

The shape of ytest:(5231,)

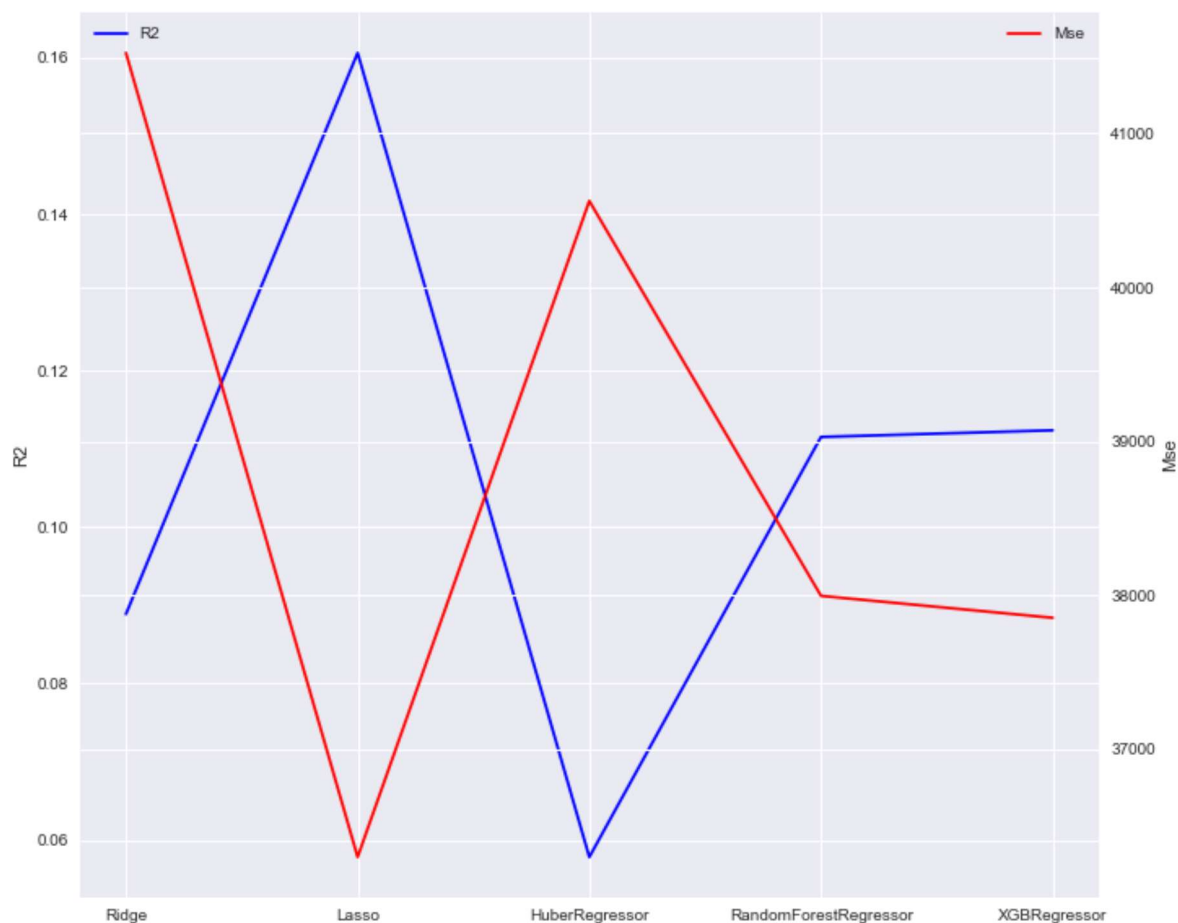
In [63]:

```
1 def r2_score(model):
2     return cross_val_score(model, X_train, y_train, cv=5)
3 def rmse_score(model):
4     return cross_val_score(model, X_train, y_train, scoring='neg_mean_squared_error', cv=5)
5
6 Model=[LinearRegression, Ridge, Lasso, HuberRegressor, RandomForestRegressor, XGBRegressor]
7 lname3=[]
8 lr23=[]
9 lrmse3=[]
10 for model in Model:
11     if model == XGBRegressor:
12         m=XGBRegressor(objective='reg:squarederror')
13     else:
14         m=model()
15     cv_r2=r2_score(m)
16     cv_rmse_score = rmse_score(m)
17     print('The mean R^2 of {} is: {:.3e}'.format(model.__name__, cv_r2.mean()))
18     print('The mean mse of {} is: {:.3e}'.format(model.__name__, -cv_rmse_score.mean()))
19     lname3.append(model.__name__)
20     lr23.append(cv_r2.mean())
21     lrmse3.append(-cv_rmse_score.mean())
22
```

```
The mean R^2 of LinearRegression is:-2.959e+09
The mean mse of LinearRegression is: 8.504e+13
The mean R^2 of Ridge is:8.887e-02
The mean mse of Ridge is: 4.152e+04
The mean R^2 of Lasso is:1.606e-01
The mean mse of Lasso is: 3.630e+04
The mean R^2 of HuberRegressor is:5.780e-02
The mean mse of HuberRegressor is: 4.056e+04
The mean R^2 of RandomForestRegressor is:1.115e-01
The mean mse of RandomForestRegressor is: 3.800e+04
The mean R^2 of XGBRegressor is:1.124e-01
The mean mse of XGBRegressor is: 3.785e+04
```

In [64]:

```
1 fig = plt.figure(figsize=(10,10))
2 ax1 = fig.add_subplot(111)
3 ax1.plot(lname3[1:], lr23[1:], 'b', label="R2");
4 ax1.legend(loc=2)
5 ax1.set_ylabel('R2');
6 ax2 = ax1.twinx() # this is the important function
7 ax2.plot(lname3[1:], lrmse3[1:], 'r', label = "Mse")
8 ax2.legend(loc=1)
9 # ax2.set_xlim([0, np.e]);
10 ax2.set_ylabel('Mse');
11 plt.show()
```



关于 Featuretools 的体会

可以看到最后经过 Featuretools 的数据集效果也不尽人意，个人感觉这一工具库还是要与人工可视分析一起使用较好，因为特征运算基元的选择不同，会导致最终生成的特征语义也不尽相同。而随意地选择运算基元，即使生成了很多特征，但很多都是没有意义的，比如 `reviews_per_month+availability_365`，没有任何的意义。

因此不应想着用这一工具库直接一劳永逸，无脑加基元，也是要人工去分析数据之间的关联，尤其是与预测变量 `y` 之间的关系。再者就是工具包无法直接自动对数据取对数或者分割区间等，库里是没有这样的基元，还是要手动去操作，这也是这个库想办法去改进的地方。

总体而言，在数据挖掘等过程中，可以在对原始数据分析的基础上利用 Featuretools 生成更深层次的特征，这样可能会比纯人工有更好的效果，但不能只依赖其进行简单的基元运算。

In []:

1	
---	--